

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

**Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці**

**Розрахунково-графічна робота
ПРОЄКТУВАННЯ ТА СПЕЦИФІКАЦІЯ
ПРОСТОЇ ІМПЕРАТИВНОЇ МОВИ ПРОГРАМУВАННЯ
ЗАГАЛЬНОГО ПРИЗНАЧЕННЯ
(з дисципліни «Основи розробки трансляторів»)
Мова програмування Krok
Рівень 2 Основний**

Учасники проєкту:

Степаненко Назар Юрійович
Аніщенко Артем

Виконав: студент III курсу, групи ТВ-43
спеціальності 121 «Інженерія програмного забезпечення»

Степаненко Назар Юрійович

(Прізвище, ім'я, по батькові)

— — — — — (Підпис)

Перевірів: доцент, к.т.н. Стативка Ю.І.

(Посада, науковий ступінь, прізвище та ініціали)

— — — — — (Підпис)

Київ – 2026

ЗМІСТ

1 Вступ	6
1.1 Назва мови	6
1.2 Рівень складності та мова-прототип	6
1.3 Призначення мови	7
1.4 Обробка програми	7
1.5 Нотація	8
1.6 Відповідність вимогам завдання	9
2 Лексика мови	11
2.1 Алфавіт	11
2.2 Лексеми	17
2.3 Коментарі та пробільні символи	18
2.4 Ідентифікатори	19
2.5 Ключові слова	19
2.6 Літерали	21
2.6.1 Цілі літерали	21
2.6.2 Дійсні літерали	21
2.6.3 Рядкові літерали	22
2.6.4 Логічні літерали	23
2.7 Оператори та роздільники	24
2.8 Автоматична вставка крапки з комою	26
2.9 Таблиця токенів	27
3 Формальний опис синтаксису	29
3.1 Структура програми	29
3.2 Оголошення	30

3.3 Типи.....	32
3.4 Функції	33
3.5 Інструкції	33
3.6 Інструкція розгалуження.....	36
3.7 Інструкція повторення.....	37
3.8 Інструкція повернення.....	37
3.9 Інструкції введення та виведення	37
3.10 Вирази	38
3.11 Властивості граматики	44
4 Семантика та система типів	47
4.1 Модель виконання	47
4.2 Типи даних.....	47
4.2.1 Вбудовані скалярні типи	47
4.2.2 Користувацькі скалярні типи	48
4.2.3 Масиви.....	48
4.2.4 Структури.....	49
4.2.5 Параметри-вказівники	50
4.2.6 Еквівалентність та сумісність типів.....	51
4.3 Оголошення, змінні та константи	51
4.4 Області видимості	52
4.5 Вирази	53
4.5.1 Пріоритет та асоціативність операторів	53
4.5.2 Порядок обчислення	54
4.5.3 Арифметичні оператори	54
4.5.4 Неявне приведення типів.....	55

4.5.5 Явне приведення типів	56
4.5.6 Оператори порівняння	56
4.5.7 Логічні оператори	57
4.5.8 Операції над рядками	57
4.5.9 Константні вирази.....	57
4.5.10 L-значення	57
4.6 Інструкції	58
4.6.1 Присвоєння.....	58
4.6.2 Блок.....	58
4.6.3 Розгалуження	58
4.6.4 Цикли.....	58
4.6.5 Повернення з функції	59
4.7 Функції	60
4.8 Введення та виведення	61
4.9 Модульність	62
4.10 Керування пам'яттю.....	62
4.11 Помилки	63
4.11.1 Помилки трансляції	63
4.11.2 Помилки часу виконання.....	64
5 Приклади програм	65
5.1 Базовий приклад	65
5.2 Типи та приведення	67
5.3 Розгалуження	67
5.4 Цикли.....	67
5.5 Функції та рекурсія.....	68

5.6 Передача параметрів за посиланням	69
5.7 Масиви та структури	69
5.8 Області видимості та затінення	71
5.9 Помилки часу виконання	71
5.10 Некоректні програми.....	72
6 Повна граматики мови.....	73

1 Вступ

1.1 Назва мови

Мова програмування, специфікація якої наведена в цьому документі, має назву **Krok** (вимовляється «крок»). Назва є транслітерацією українського слова «крок» і пов'язана з мовою-прототипом: англійське *go* означає «іди», а крок — елементарна одиниця руху. Так само кожна інструкція програми є одним кроком алгоритму, а сама мова спроектована як «крок» від повноцінної мови Go до простої навчальної мови, для якої в межах дисципліни буде розроблено транслятор.

Файли з вихідним кодом мовою Krok мають розширення `.krok`.

1.2 Рівень складності та мова-прототип

Мова Krok відповідає **Рівню 2 (Основний)**: процедурно-модульна статично типізована імперативна мова з користувацькими типами. Об'єктно-орієнтовані концепції не підтримуються.

Мовою-прототипом обрано мову **Go** (https://go.dev/doc/effective_go). З мови Go запозичено: синтаксис оголошень (`var`, `const`, `type`, `func`), структуру програми (`package main`, `import "fmt"`, функція `main`), інструкцію присвоєння (`=`) та оголошення з ініціалізацією (`:=`), інкремент і декремент (`++`, `--`), інструкції повторення `for` та розгалуження `if-else`, введення/виведення через пакет `fmt`, однорядкові коментарі `//`, правило автоматичної вставки крапки з комою, масиви, структури та вказівники як механізм передачі параметрів за посиланням.

Семантику елементів мови адаптовано до потреб навчального транслятора. Основні відмінності від Go наведено в табл. 1.1.

Таблиця 1.1 – Основні відмінності мови Krok від мови-прототипу Go

Аспект	Go	Krok
Приведення <code>int</code> → <code>float</code>	лише явне	неявне розширення та явне приведення
Піднесення до степеня	немає оператора	оператор <code>**</code> , правоасоціативний

Аспект	Go	Krok
Переповнення int	циклічне (wrap-around)	помилка часу виконання
Типи int, float, bool, string	попередньо оголошені ідентифікатори	ключові слова
Вказівники	довільні значення, nil	лише типи параметрів функцій
Складені літерали	для масивів, зрізів, структур, мап	лише для масивів із явним типом [N]T
Інструкція for	чотири форми, break, continue	дві форми: з передумовою та трикомпонентна
if з ініціалізацією, switch	є	немає
Зрізи, мапи, методи, інтерфейси, горутини	є	немає
Пакети	довільні	лише main та вбудований fmt

Проект виконується спільно (виконавці — Степаненко Н. Ю. та Аніщенко А.). Відповідно до вимог до спільних проєктів кожен виконавець обирає власний варіант інструкцій повторення та розгалуження. У цій специфікації описано варіант: **інструкція for з передумовою та трикомпонентна інструкція for** (розд. 3.7) і **інструкція if з необов'язковими гілками else if та else** (розд. 3.6).

1.3 Призначення мови

Krok — мова загального призначення для запису обчислювальних алгоритмів, що працюють зі стандартними потоками введення та виведення: числові розрахунки, обробка масивів, рекурсивні алгоритми, прості моделі даних на основі структур. Мова навмисно компактна: її граматики належить до класу LL(1), кожна конструкція має одне прочитання, а семантичні правила перевіряються статично на етапі трансляції, що робить мову придатною для реалізації лексичного, синтаксичного та семантичного аналізаторів у межах навчального курсу.

1.4 Обробка програми

Програма мовою Krok подається на вхід транслятора (компілятора або інтерпретатора) у вигляді одного текстового файлу в кодуванні UTF-8. Трансляція складається з таких фаз:

1. Лексичний аналіз — окремий прохід, що перетворює послідовність символів на послідовність токенів, вилючає коментарі та пробільні символи і вставляє неявні крапки з комою (розд. 2.8). Уся інформація, необхідна для побудови лексичного аналізатора, міститься в розд. 2.
2. Синтаксичний аналіз — перевірка відповідності послідовності токенів граматиці з розд. 3 та побудова дерева розбору. Граматика належить до класу LL(1), тож аналіз можна виконати методом рекурсивного спуску з одним токеном попереднього перегляду.
3. Семантичний аналіз — перевірка контекстних умов: оголошеність імен, області видимості, сумісність типів тощо (розд. 4).
4. Генерування коду або безпосереднє виконання.

Результат трансляції виконується системою часу виконання (run-time system), яка отримує вхідні дані зі стандартного потоку введення, записує результати у стандартний потік виведення, а повідомлення про помилки — у стандартний потік помилок.

1.5 Нотація

Для опису мови Krok використовується розширена нотація Бекуса — Наура (РБНФ) у варіанті Н. Вірта. Метасимволи нотації наведено в табл. 1.2.

Таблиця 1.2 – Прийнята нотація РБНФ

Метасимвол	Значення
$A = \alpha$	нетермінал A визначається як α
$\alpha \mid \beta$	альтернатива: α або β
$[\alpha]$	0 або 1 екземпляр α
$\{ \alpha \}$	0 або більше екземплярів α
$(\alpha \mid \beta)$	групування
Abc	нетермінал (починається з великої літери)
abc	термінал — ключове слово (починається з малої літери)
'+', 'Scan'	термінал, записаний у лапках буквально
'\t', '\n', '\r'	термінали: символи табуляції, нового рядка та повернення каретки
? текст ?	спеціальна послідовність: неформально описана множина символів
(* текст *)	коментар до граматики

Правило починається з імені нетермінала на початку рядка; рядки, що починаються з пробілів, продовжують попереднє правило. Кожне правило має лише один нетермінал у лівій частині, отже граматики мови є контекстно-вільною.

Граматику мови поділено на дві частини. **Лексична граматика** (аксіома SourceText) описує, як із символів алфавіту утворюються лексеми. **Синтаксична граматика** (аксіома Program) описує, як із токенів утворюються програми; нетермінали Ident, IntLit, FloatLit та StringLit у синтаксичній граматиці є неподільними токенами, визначеними лексичною граматикою. Нетермінали BoolLit, RelOp, AssignOp та IncDecOp групують токени і використовуються в обох частинах.

Для графічного представлення граматики використовуються синтаксичні діаграми Вірта. Діаграми згенеровано автоматично з файлу граматики grammar/krok.ebnf програмою tools/gen_diagrams.py (бібліотека railroad-diagrams), тому кожне правило має рівно одну діаграму з ідентичними назвами символів. На діаграмах термінали зображено в прямокутниках із заокругленими кутами, нетермінали — у прямокутниках (курсивом), спеціальні послідовності — курсивом без рамки.

1.6 Відповідність вимогам завдання

Таблиця 1.3 – Відповідність специфікації вимогам завдання

Вимога	Конструкція мови Krok	Розділ
Оператор циклу	for умова { }, for init; умова; post { }	3.7
Оператор розгалуження	if умова { } else if умова { } else { }	3.6
Цілі та дійсні числа, + - * /	типи int, float; оператори + - * / %	4.2.1, 4.5.3
Піднесення до степеня (правоасоціативне)	оператор **	4.5.1
Логічний тип	bool, true, false, && !	4.2.1, 4.5.7
Рядковий тип	string, рядкові літерали	4.2.1, 4.5.8
Іменовані константи	const	4.3
Порівняння	== != < <= > >=	4.5.6
Дужки у виразах	(Expression)	3.10
Однорядкові коментарі	// ...	2.3

Вимога	Конструкція мови Krok	Розділ
Приведення <code>int</code> ↔ <code>float</code>	неявне розширення, <code>float(x)</code> , <code>int(x)</code>	4.5.4, 4.5.5
Введення/виведення	<code>fmt.Scan</code> , <code>fmt.Print</code> , <code>fmt.Println</code>	4.8
Функції, рекурсія	<code>func</code> , <code>return</code>	4.7
Передача за значенням і за посиланням	параметри <code>T</code> та <code>*T</code> , <code>&x</code> , <code>*p</code>	4.2.5
Локальні та глобальні змінні	блоки, затінення	4.4
Користувачькі скалярні типи	<code>type Celsius float</code>	4.2.2
Масиви (багатовимірні, статична та динамічна ініціалізація)	<code>[N]T</code> , <code>[N]T{...}</code>	4.2.3
Структури (факультативно)	<code>type P struct { ... }</code>	4.2.4
Модульна структура (факультативно)	<code>import "fmt"</code>	4.9

2 Лексика мови

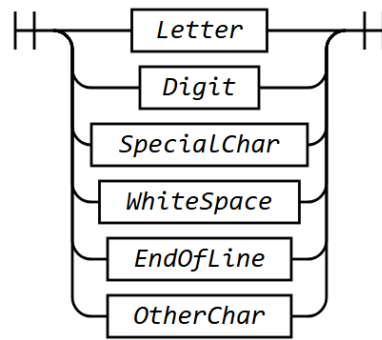
Лексичний аналіз виконується окремим проходом і не залежить від синтаксичних та семантичних аспектів мови. Лексичний аналізатор читає текст програми зліва направо і на кожному кроці виділяє **найдовшу** послідовність символів, що утворює лексему (правило найдовшого збігу). Наприклад, ****** розпізнається як один оператор піднесення до степеня, а не як два оператори множення; **<=** — як один оператор, а **a---b** — як **a**, **--**, **-**, **b**.

2.1 Алфавіт

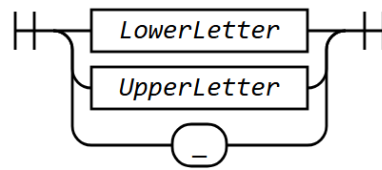
Текст програми мовою Krok записується в кодуванні UTF-8. Поза рядковими літералами та коментарями програма може містити лише латинські літери, цифри, спеціальні знаки, пробільні символи та символи кінця рядка. Усередині рядкових літералів і коментарів допускаються будь-які символи Unicode (зокрема кирилиця), крім символів кінця рядка. Мова чутлива до регістру літер: **total**, **Total** і **TOTAL** — різні ідентифікатори.

Alphabet	= Letter Digit SpecialChar WhiteSpace EndOfLine OtherChar
Letter	= LowerLetter UpperLetter '_'
LowerLetter	= 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o' 'p' 'q' 'r' 's' 't' 'u' 'v' 'w' 'x' 'y' 'z'
UpperLetter	= 'A' 'B' 'C' 'D' 'E' 'F' 'G' 'H' 'I' 'J' 'K' 'L' 'M' 'N' 'O' 'P' 'Q' 'R' 'S' 'T' 'U' 'V' 'W' 'X' 'Y' 'Z'
Digit	= '0' NonZeroDigit
NonZeroDigit	= '1' '2' '3' '4' '5' '6' '7' '8' '9'
SpecialChar	= '+' '-' '*' '/' '%' '=' '!' '<' '>' '&' ' ' ':' ';' ',' '.' '(' ')' '[' ']' '{' '}' '"' '\'
WhiteSpace	= ' ' '\t'
EndOfLine	= '\n' '\r' '\n'
OtherChar	= ? інший символ Unicode (лише в StringLit та Comment) ?

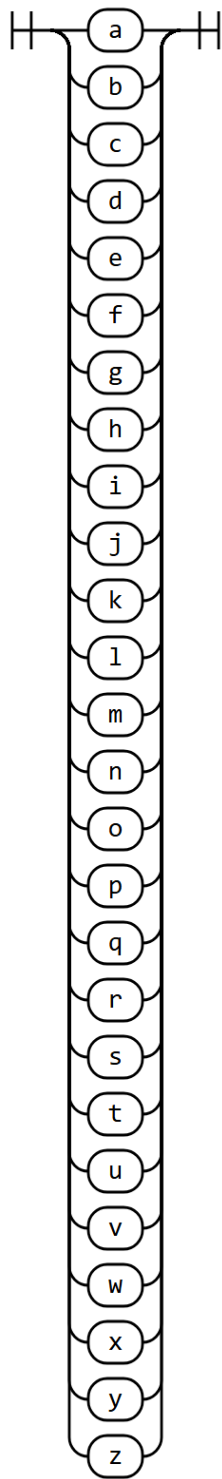
Alphabet



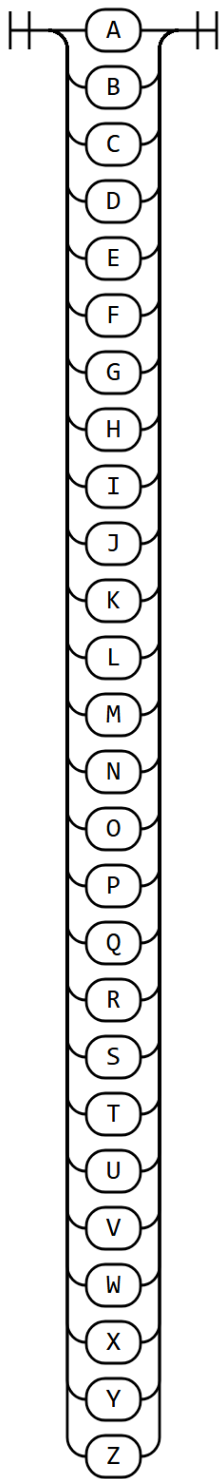
Letter



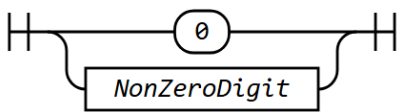
LowerLetter



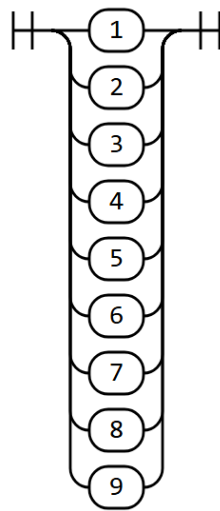
UpperLetter



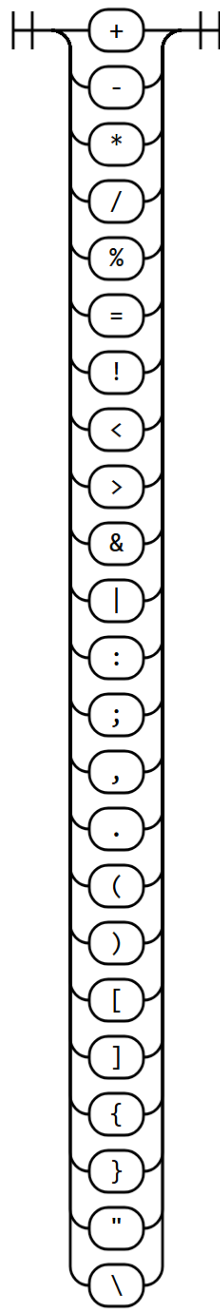
Digit



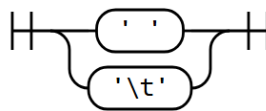
NonZeroDigit



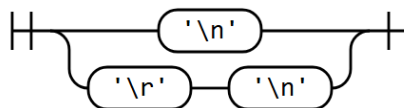
SpecialChar



WhiteSpace



EndOfLine



OtherChar

 ? *інший символ Unicode (лише в StringLit та Comment)* ? 

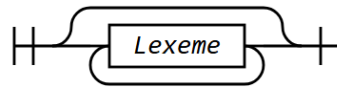
Рисунок 2.1 – Синтаксичні діаграми алфавіту

2.2 Лексеми

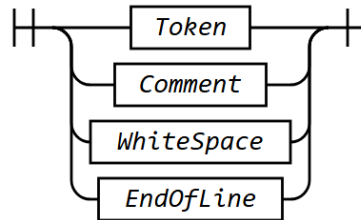
Текст програми є послідовністю лексем. Токени (ідентифікатори, ключові слова, літерали, оператори та роздільники) передаються синтаксичному аналізатору; коментарі, пробільні символи та символи кінця рядка лише розділяють токени й ігноруються. Виняток — символ кінця рядка, після якого за правилом розд. 2.8 може бути вставлено токен `;`. Між двома токенами, які інакше злилися б в один (наприклад, ідентифікатор і ключове слово), має бути принаймні один пробільний символ, символ кінця рядка або коментар.

```
SourceText    = { Lexeme }  
Lexeme        = Token | Comment | WhiteSpace | EndOfLine  
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit  
              | Operator | Delimiter
```

SourceText



Lexeme



Token

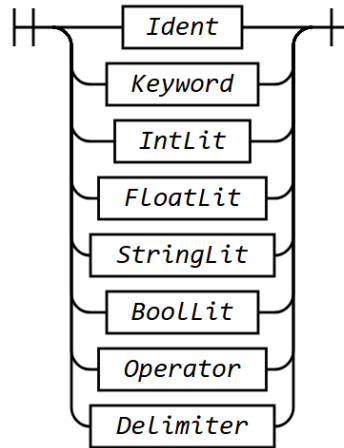


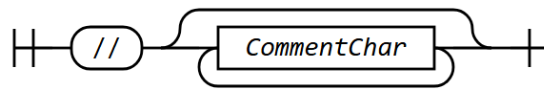
Рисунок 2.2 – Синтаксичні діаграми лексем

2.3 Коментарі та пробільні символи

Однорядковий коментар починається парою символів `//` поза рядковим літералом і продовжується до кінця рядка. Символ кінця рядка до коментаря не належить. Коментар вилучається лексичним аналізатором і діє як пробіл. Послідовність `//` усередині рядкового літерала коментарем не є.

```
Comment      = '//' { CommentChar }  
CommentChar  = ? будь-який символ Unicode, крім '\n' та '\r' ?
```

Comment



CommentChar

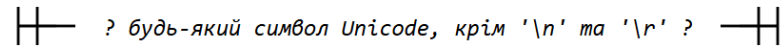


Рисунок 2.3 – Синтаксичні діаграми коментаря

Приклад:

```
x := 10 // коментар до кінця рядка  
s := "// це не коментар"
```

2.4 Ідентифікатори

Ідентифікатори позначають змінні, константи, типи, поля структур і функції. Ідентифікатор починається з літери або символу підкреслення, за якими можуть іти літери, підкреслення та цифри. Довжина ідентифікатора не обмежена.

```
Ident      = Letter { Letter | Digit }
```

Ident

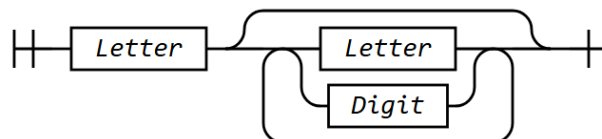


Рисунок 2.4 – Синтаксична діаграма ідентифікатора

Якщо лексема, що за формою могла б бути ідентифікатором, збігається з ключовим словом (розд. 2.5), вона є ключовим словом. Якщо вона збігається зі словом `true` або `false`, вона є логічним літералом.

Приклади: `x`, `total`, `Point`, `_tmp1`, `matrix2D`.

2.5 Ключові слова

Ключові слова зарезервовані й не можуть використовуватися як ідентифікатори.

Keyword	=	package		import		func		var		const		type		struct			
			if		else		for		return		int		float		bool		string
			len		fmt		'Print'		'Println'		'Scan'						

Keyword

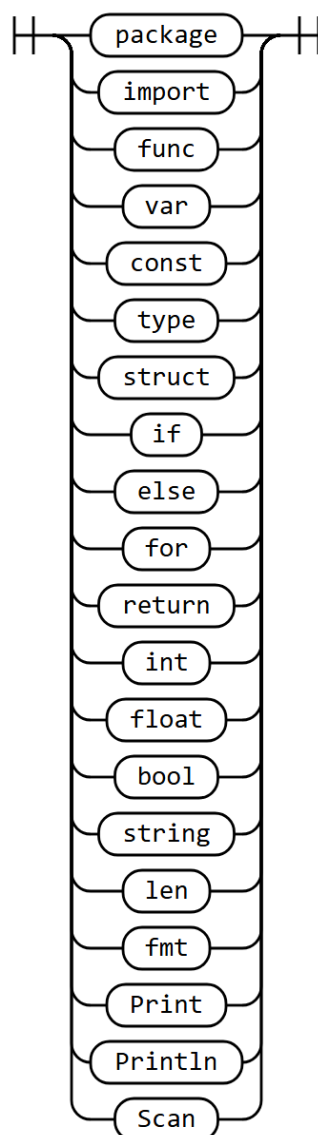


Рисунок 2.5 – Синтаксична діаграма ключових слів

Слова `int`, `float`, `bool` та `string` позначають вбудовані типи, `len` — вбудовану функцію довжини. Слово `fmt` позначає вбудований пакет введення/виведення, а `Print`, `Println` і `Scan` — його операції. Слова `true` та `false` є логічними літералами (розд. 2.6).

2.6 Літерали

Літерали позначають сталі значення. Тип літерала визначається його формою. Знак числа до літерала не належить: у записі `-5` лексичний аналізатор виділяє два токени, оператор `-` і літерал `5`, а унарний мінус розпізнається під час синтаксичного аналізу.

2.6.1 Цілі літерали

Цілий літерал — десяткова послідовність цифр без знака. Щоб уникнути плутанини з вісімковими літералами мови Go, запис із ведучим нулем (наприклад, `007`) є лексичною помилкою; єдиний цілий літерал, що починається з `0`, — сам `0`. Цілий літерал має тип `int`, а його значення не може перевищувати `9223372036854775807`; інакше виникає помилка трансляції CE-10.

```
IntLit      = '0' | NonZeroDigit { Digit }
```

IntLit

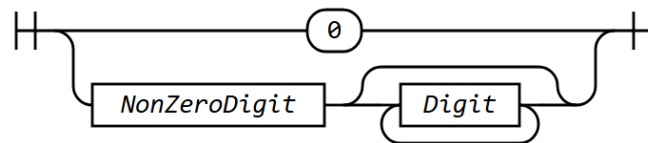


Рисунок 2.6 – Синтаксична діаграма цілого літерала

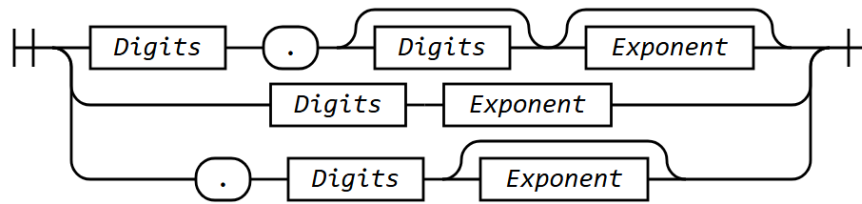
Приклади: `0`, `7`, `42`, `9223372036854775807`.

2.6.2 Дійсні літерали

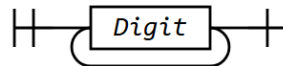
Дійсний літерал має дробову частину, експоненту або обидві. Цілу або дробову частину (але не обидві) можна пропустити. Дійсний літерал має тип `float`; значення, що після округлення до найближчого значення типу `float` дає нескінченність, є помилкою трансляції CE-10.

```
FloatLit    = Digits '.' [ Digits ] [ Exponent ]
              | Digits Exponent
              | '.' Digits [ Exponent ]
Digits      = Digit { Digit }
Exponent    = ( 'e' | 'E' ) [ '+' | '-' ] Digits
```

FloatLit



Digits



Exponent

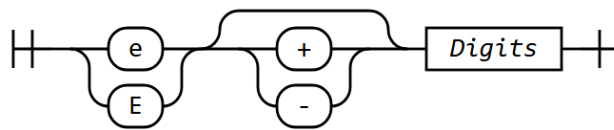


Рисунок 2.7 – Синтаксичні діаграми дійсного літерала

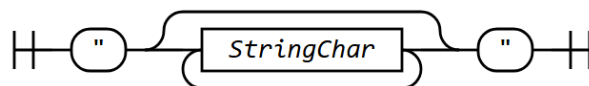
Приклади: 3.14, 2., .5, 1e3, 6.02E+23, 1.5e-3, 007.5.

2.6.3 Рядкові літерали

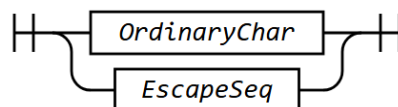
Рядковий літерал — послідовність символів у подвійних лапках. Він не може містити символ кінця рядка. Порожній рядок записується як "". Для запису спеціальних символів використовуються escape-послідовності (табл. 2.1); будь-яка інша послідовність, що починається зі зворотної косої риски, є лексичною помилкою CE-01.

```
StringLit    = '"' { StringChar } '"'
StringChar   = OrdinaryChar | EscapeSeq
EscapeSeq    = '\' ( 'n' | 't' | 'r' | '"' | '\' )
OrdinaryChar = ? будь-який символ Unicode, крім '"', '\', '\n' та '\r' ?
```

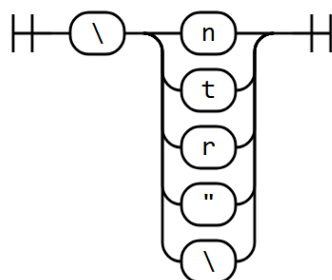
StringLit



StringChar



EscapeSeq



OrdinaryChar

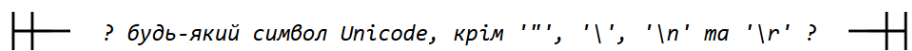


Рисунок 2.8 – Синтаксичні діаграми рядкового літерала

Таблиця 2.1 – Escape-послідовності рядкових літералів

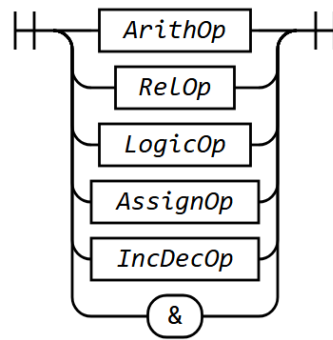
Послідовність	Символ	Код Unicode
<code>\n</code>	новий рядок	U+000A
<code>\t</code>	горизонтальна табуляція	U+0009
<code>\r</code>	повернення каретки	U+000D
<code>\"</code>	подвійні лапки	U+0022
<code>\\</code>	зворотна коса риска	U+005C

Приклади: `"`, `"Krok"`, `"Привіт, світе!"`, `"стовпчик 1\tстовпчик 2\n"`,
"лапки: `\\"x\"`", коса риска: `\\`".

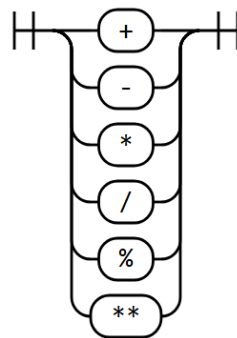
2.6.4 Логічні літерали

Логічні літерали `true` та `false` мають тип `bool`. Правило `BoolLit` наведено в розд. 3.10.

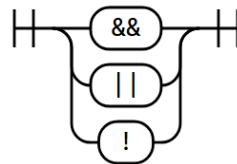
Operator



ArithOp



LogicOp



Delimiter

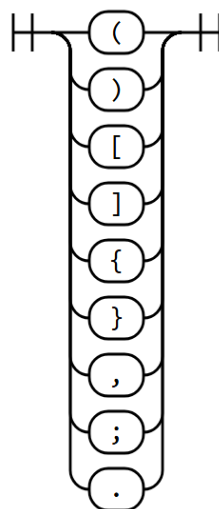


Рисунок 2.9 – Синтаксичні діаграми операторів і роздільників

Лексеми `+`, `-` та `*` мають кілька ролей, які розрізняє синтаксичний аналізатор за контекстом: `+` і `-` — бінарні або унарні оператори, `*` — множення, розіменування (`*p`) або позначка типу параметра-вказівника (`*int`).

2.8 Автоматична вставка крапки з комою

Граматика мови використовує токен `;` як завершувач оголошень та інструкцій. Як і в мові Go, у тексті програми крапку з комою зазвичай не пишуть: її вставляє лексичний аналізатор за таким правилом.

Коли лексичний аналізатор зустрічає символ кінця рядка або кінець файлу, він вставляє в потік токенів `;`, якщо останній виділений у цьому рядку токен є одним із таких:

- ідентифікатор;
- цілий, дійсний, рядковий або логічний літерал;
- ключове слово `return`, `int`, `float`, `bool` або `string`;
- оператор `++` чи `--`;
- закриваюча дужка `)`, `]` або `}`.

Коментар до кінця рядка на правило не впливає. Явно записана крапка з комою дозволяє розмістити кілька інструкцій в одному рядку й обов'язкова в заголовку трикомпонентної інструкції `for`.

Наслідки правила:

- фігурна дужка, що відкриває блок, повинна стояти в тому самому рядку, що й `if`, `for`, `func` чи `struct`;
- ключове слово `else` повинно стояти в тому самому рядку, що й закриваюча дужка `}` попереднього блоку, інакше після `}` буде вставлено `;` і виникне синтаксична помилка;
- список елементів літерала масиву, розбитий на кілька рядків, має закінчуватися комою перед переходом на новий рядок.

```
if x > 0 {           // правильно
    y = 1
} else {            // правильно: else у рядку з }
    y = 2
}
for i := 0; i < 3; i++ { fmt.Print(i); fmt.Print(" ") } // явні ;
```

2.9 Таблиця токенів

Перелік усіх лексем мови Krok наведено в табл. 2.2. Один приклад лексеми в рядку означає, що лексема унікальна; текст «символ ...» означає, що лексема є термінальним символом граматики.

Таблиця 2.2 – Таблиця лексем мови Krok

Код	Приклади лексем	Токен	Неформальний опис
1	x, total, Point, tmp1	id	ідентифікатор
2	0, 42, 1000	intnum	ціле без знака
3	3.14, 2., .5, 1e3, 6.02E+23	floatnum	дійсне без знака
4	"", "Krok", "a\tb\n"	strlit	рядковий літерал
5	true	boolval	логічне значення
6	false	boolval	логічне значення
7	package	keyword	символ package
8	import	keyword	символ import
9	func	keyword	символ func
10	var	keyword	символ var
11	const	keyword	символ const
12	type	keyword	символ type
13	struct	keyword	символ struct
14	if	keyword	символ if
15	else	keyword	символ else
16	for	keyword	символ for
17	return	keyword	символ return
18	int	keyword	символ int
19	float	keyword	символ float
20	bool	keyword	символ bool
21	string	keyword	символ string
22	len	keyword	символ len
23	fmt	keyword	символ fmt
24	Print	keyword	символ Print
25	Println	keyword	символ Println
26	Scan	keyword	символ Scan
27	+	add_op	додавання, унарний плюс, конкатенація
28	-	add_op	віднімання, унарний мінус
29	*	mult_op	множення; розіменування
30	/	mult_op	ділення
31	%	mult_op	остача від ділення
32	**	pow_op	піднесення до степеня
33	==	rel_op	дорівнює
34	!=	rel_op	не дорівнює
35	<	rel_op	менше
36	<=	rel_op	менше або дорівнює
37	>	rel_op	більше
38	>=	rel_op	більше або дорівнює

Код	Приклади лексем	Токен	Неформальний опис
39	&&	and_op	логічне «і»
40		or_op	логічне «або»
41	!	not_op	логічне заперечення
42	=	assign_op	присвоєння
43	:=	define_op	оголошення з ініціалізацією
44	++	incdec_op	інкремент
45	--	incdec_op	декремент
46	&	addr_op	взяття адреси
47	(	paren	ліва кругла дужка
48	)	paren	права кругла дужка
49	[	bracket	ліва квадратна дужка
50	]	bracket	права квадратна дужка
51	{	brace	ліва фігурна дужка
52	}	brace	права фігурна дужка
53	,	punct	кома
54	;	punct	крапка з комою (явна або вставлена)
55	.	punct	крапка (доступ до поля, fmt.)
56	// коментар	comment	однорядковий коментар (ігнорується)
57	\32	ws	пробіл (ігнорується)
58	\t	ws	горизонтальна табуляція (ігнорується)
59	\n, \r\n	eol	кінець рядка (може породити ;)

3 Формальний опис синтаксису

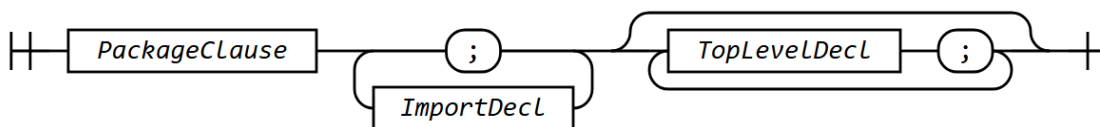
У цьому розділі наведено синтаксичну граматику мови. Нетермінали `Ident`, `IntLit`, `FloatLit` та `StringLit` є токенами (розд. 2), токен `;` може бути явним або вставленим (розд. 2.8).

3.1 Структура програми

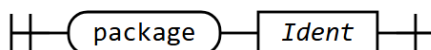
Програма мовою Krok складається з одного файлу. Файл починається оголошенням пакета `package main`, за яким іде нуль або більше оголошень імпорту та нуль або більше оголошень верхнього рівня: змінних, констант, типів і функцій.

```
Program      = PackageClause ';' { ImportDecl ';' } { TopLevelDecl ';' }  
PackageClause = package Ident  
ImportDecl   = import StringLit  
TopLevelDecl = VarDecl | ConstDecl | TypeDecl | FuncDecl
```

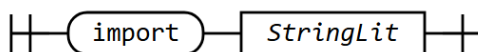
Program



PackageClause



ImportDecl



TopLevelDecl

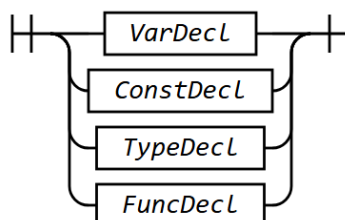


Рисунок 3.1 – Синтаксичні діаграми структури програми

Приклад:

```
package main

import "fmt"

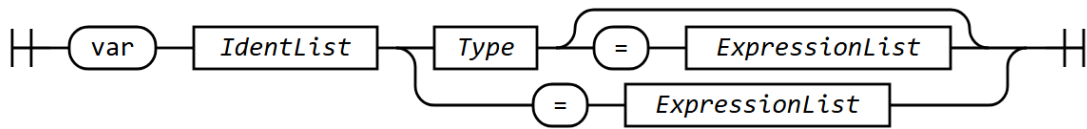
func main() {
    fmt.Println("Привіт, Krok!")
}
```

3.2 Оголошення

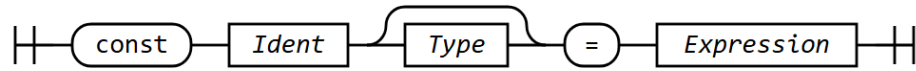
Оголошення вводять імена змінних, констант і типів. Змінні оголошуються ключовим словом `var`, константи — `const`, типи — `type`. Синтаксично оголошення змінних і констант відрізняються ключовим словом, а також тим, що константа завжди має ініціалізатор і оголошує рівно одне ім'я.

```
VarDecl      = var IdentList
              ( Type [ '=' ExpressionList ] | '=' ExpressionList )
ConstDecl    = const Ident [ Type ] '=' Expression
TypeDecl     = type Ident ( Type | StructType )
StructType   = struct '{' FieldList '}'
FieldList    = [ FieldDecl FieldTail ]
FieldTail    = [ ';' FieldList ]
FieldDecl    = IdentList Type
IdentList    = Ident { ',' Ident }
```

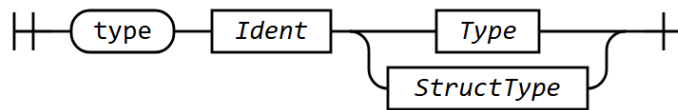
VarDecl



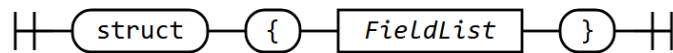
ConstDecl



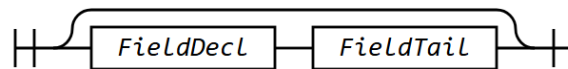
TypeDecl



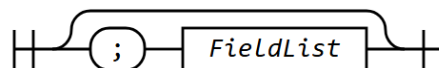
StructType



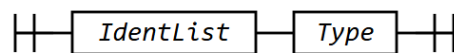
FieldList



FieldTail



FieldDecl



IdentList

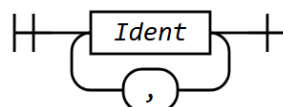


Рисунок 3.2 – Синтаксичні діаграми оголошень

Приклади:

```

var count int
var x, y float = 1.5, -2
var name = "Krok"
const Max int = 100
const Pi = 3.14159
type Celsius float
type Point struct {
    x, y float
    label string
}

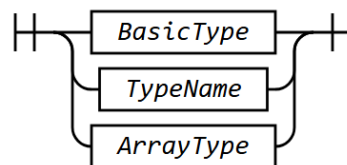
```

3.3 Типи

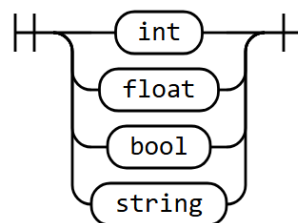
Тип позначається вбудованим ім'ям, іменем користувацького типу або описом типу масиву. Опис структури (*struct*) може стояти лише в оголошенні типу, а позначка вказівника (*) — лише в типі параметра функції (розд. 3.4); ці обмеження виражено граматикою.

Type	= BasicType TypeName ArrayType
BasicType	= int float bool string
TypeName	= Ident
ArrayType	= '[' Expression ']' Type

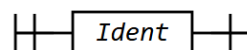
Type



BasicType



TypeName



ArrayType

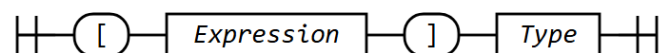


Рисунок 3.3 – Синтаксичні діаграми типів

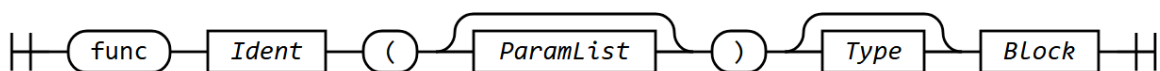
Приклади: `int`, `string`, `Celsius`, `[10]float`, `[3][4]int`, `[Size]Point`.

3.4 Функції

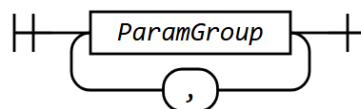
Оголошення функції задає її ім'я, список параметрів, необов'язковий тип результату та тіло. Кілька послідовних параметрів одного типу можна згрупувати: `a, b int`.

```
FuncDecl    = func Ident '(' [ ParamList ] ')' [ Type ] Block
ParamList   = ParamGroup { ',', ParamGroup }
ParamGroup  = IdentList ParamType
ParamType   = [ '*' ] Type
```

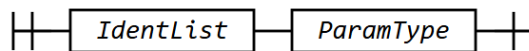
FuncDecl



ParamList



ParamGroup



ParamType



Рисунок 3.4 – Синтаксичні діаграми оголошення функції

Приклади:

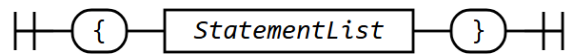
```
func square(x float) float { return x * x }
func swap(a, b *int) { t := *a; *a = *b; *b = t }
func main() { }
```

3.5 Інструкції

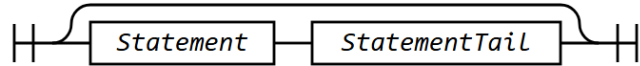
Інструкції визначають алгоритмічні дії. Послідовність інструкцій записується в блоці у фігурних дужках; інструкції розділяються токеном `;`. Оголошення змінних і констант усередині блоку також є інструкціями.

```
Block      = '{' StatementList '}'
StatementList = [ Statement StatementTail ]
StatementTail = [ ';' StatementList ]
Statement    = VarDecl | ConstDecl | IfStmt | ForStmt | ReturnStmt
              | IoStmt | Block | SimpleStmt
SimpleStmt   = Expression [ AssignOp Expression | IncDecOp ]
AssignOp     = '=' | ':='
IncDecOp     = '++' | '--'
```

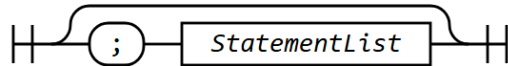
Block



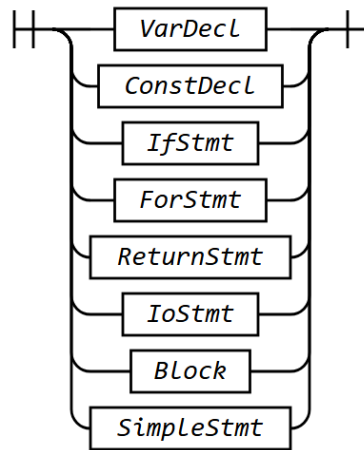
StatementList



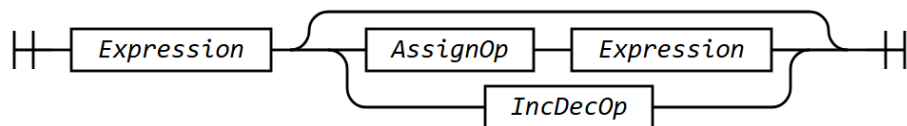
StatementTail



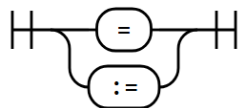
Statement



SimpleStmt



AssignOp



IncDecOp

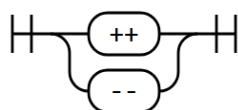


Рисунок 3.5 – Синтаксичні діаграми блоку та інструкцій

Проста інструкція `SimpleStmt` починається з виразу, після якого може стояти оператор присвоєння або оголошення з ініціалізацією та другий вираз, або оператор інкременту чи декременту. Граматика допускає ширшу множину ланцюжків, ніж мова: вимоги до лівої частини присвоєння та до інструкції-виразу є контекстними умовами (розд. 4.6.1). Це дозволяє розрізняти присвоєння, оголошення і виклик функції з одним токеном попереднього перегляду.

Приклади: `x = 5`, `a[i] = a[i] * 2`, `p.x = 1.0`, `*ptr = 0`, `total := 0`, `i++`, `k--`, `swap(&a, &b)`.

3.6 Інструкція розгалуження

Інструкція `if` виконує блок, якщо умова істинна. Необов'язкова гілка `else` містить або блок, або наступну інструкцію `if`, що утворює ланцюжок `else if`.

```
IfStmt      = if Expression Block [ else ( IfStmt | Block ) ]
```

IfStmt

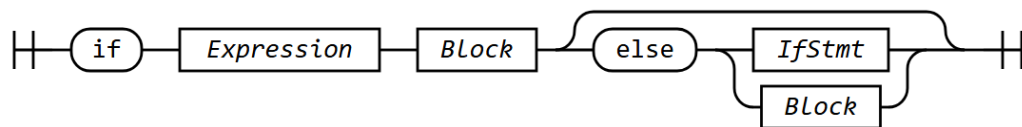


Рисунок 3.6 – Синтаксична діаграма інструкції розгалуження

Фігурні дужки навколо гілок обов'язкові. Завдяки цьому граматика не має проблеми «висячого `else`»: кожне `else` однозначно належить до `if`, блок якого закривається безпосередньо перед ним.

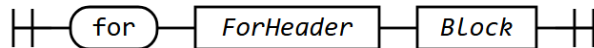
```
if t < 0 {
    fmt.Println("мороз")
} else if t < 15 {
    fmt.Println("прохолодно")
} else {
    fmt.Println("тепло")
}
```

3.7 Інструкція повторення

Інструкція `for` має дві форми. **Цикл із передумовою** містить після `for` лише умову. **Трикомпонентний цикл** (цикл із лічильником) містить ініціалізацію, умову та інструкцію наступного кроку, розділені явними `;`.

```
ForStmt      = for ForHeader Block
ForHeader    = SimpleStmt [ ';' Expression ';' SimpleStmt ]
```

ForStmt



ForHeader

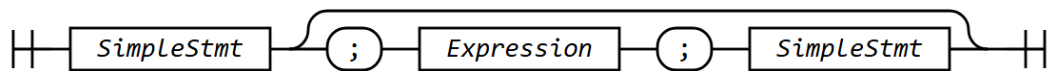


Рисунок 3.7 – Синтаксичні діаграми інструкції повторення

```
for n > 0 {                // цикл із передумовою
    n = n / 10
}
for i := 0; i < 10; i++ {   // трикомпонентний цикл
    fmt.Print(i, " ")
}
```

3.8 Інструкція повернення

```
ReturnStmt   = return [ Expression ]
```

ReturnStmt

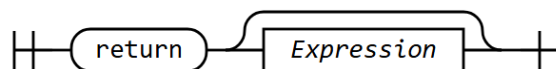


Рисунок 3.8 – Синтаксична діаграма інструкції повернення

Приклади: `return`, `return n * factorial(n-1)`.

3.9 Інструкції введення та виведення

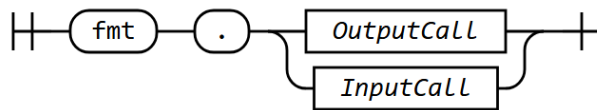
Введення та виведення виконуються операціями вбудованого пакета `fmt`. Аргументами `Scan` можуть бути лише адреси змінних (вирази з оператором `&`), що виражено граматикою.

```

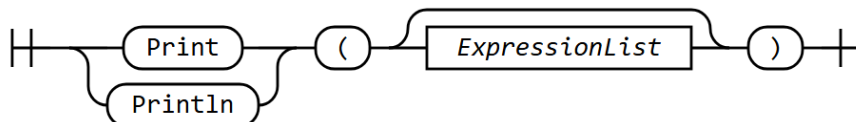
IoStmt      = fmt '.' ( OutputCall | InputCall )
OutputCall  = ( 'Print' | 'Println' ) '(' [ ExpressionList ] ')'
InputCall   = 'Scan' '(' AddrExpr { ',' AddrExpr } ')'

```

IoStmt



OutputCall



InputCall

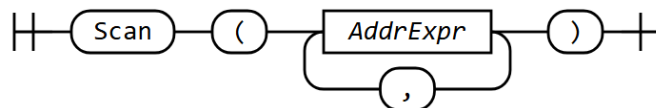


Рисунок 3.9 – Синтаксичні діаграми інструкцій введення та виведення

Приклади: `fmt.Print("x = ", x)`, `fmt.Println(a, b, a + b)`,
`fmt.Println(), fmt.Scan(&n, &v[i], &p.x)`.

3.10 Вирази

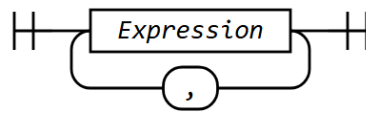
Вираз — послідовність операторів і операндів, що визначає обчислення значення. Пріоритет операторів задано стратифікацією граматики: оператори з вищим пріоритетом описано на глибших рівнях вкладеності (`OrExpr` → `AndExpr` → `RelExpr` → `AddExpr` → `MulExpr` → `UnaryExpr` → `PowerExpr` → `PrimaryExpr`). Лівоасоціативні оператори описано повтореннями { ... }, правоасоціативний оператор `**` — правою рекурсією через `UnaryExpr`. Оператори порівняння неасоціативні: запис `a < b < c` є синтаксичною помилкою.

```

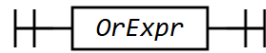
ExpressionList = Expression { ',' Expression }
Expression     = OrExpr
OrExpr         = AndExpr { '||' AndExpr }
AndExpr        = RelExpr { '&&' RelExpr }
RelExpr        = AddExpr [ RelOp AddExpr ]
AddExpr        = MulExpr { AddOp MulExpr }
MulExpr        = UnaryExpr { MulOp UnaryExpr }
UnaryExpr      = UnaryOp UnaryExpr | PowerExpr
PowerExpr      = PrimaryExpr [ '**' UnaryExpr ]

```

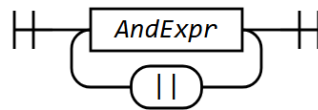
ExpressionList



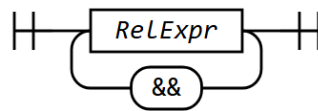
Expression



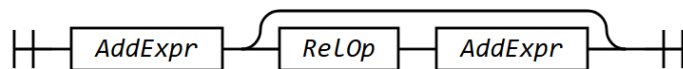
OrExpr



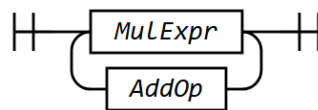
AndExpr



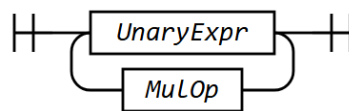
RelExpr



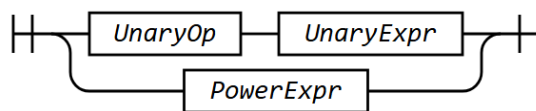
AddExpr



MulExpr



UnaryExpr



PowerExpr

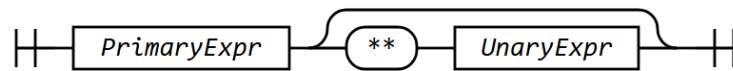
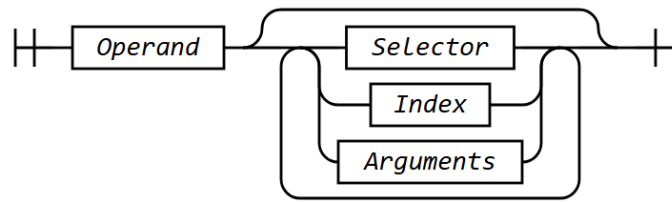


Рисунок 3.10 – Синтаксичні діаграми виразів

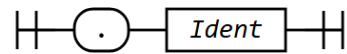
Первинний вираз складається з операнда та нуля або більше суфіксів: доступу до поля (**.**x), індексування (**[i]**) та виклику (**((args))**). Операндом може бути літерал, ідентифікатор, розіменування параметра-вказівника, явне приведення до вбудованого типу, виклик **len**, літерал масиву або вираз у дужках.

```
PrimaryExpr    = Operand { Selector | Index | Arguments }
Selector       = '.' Ident
Index          = '[' Expression ']'
Arguments      = '(' [ ArgumentList ] ')'
ArgumentList   = Argument { ',' Argument }
Argument       = AddrExpr | Expression
AddrExpr      = '&' PrimaryExpr
Operand        = Literal | Ident | Dereference | Conversion | LenCall
               | ArrayLit | '(' Expression ')'
Dereference    = '*' Ident
Conversion     = BasicType '(' Expression ')'
LenCall        = len '(' Expression ')'
```

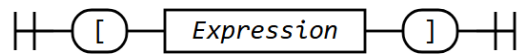
PrimaryExpr



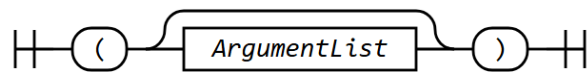
Selector



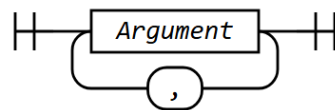
Index



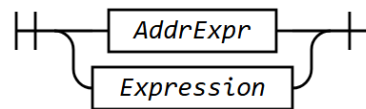
Arguments



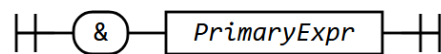
ArgumentList



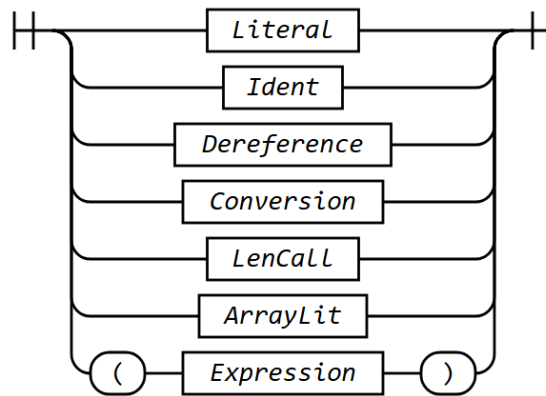
Argument



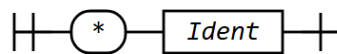
AddrExpr



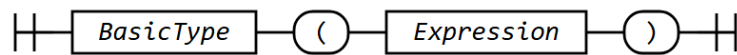
Operand



Dereference



Conversion



LenCall

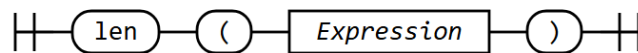
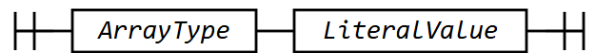


Рисунок 3.11 – Синтаксичні діаграми первинних виразів

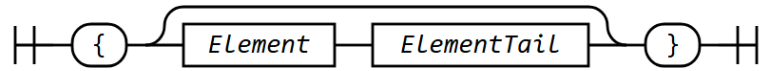
Літерал масиву складається з типу масиву та списку значень елементів у фігурних дужках. Для вкладених масивів тип внутрішніх списків не записується. Після останнього елемента допускається кома.

```
ArrayLit      = ArrayType LiteralValue
LiteralValue  = '{' [ Element ElementTail ] '}'
ElementTail   = [ ',' [ Element ElementTail ] ]
Element       = Expression | LiteralValue
Literal       = IntLit | FloatLit | StringLit | BoolLit
BoolLit       = true | false
```

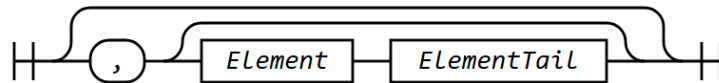
ArrayLit



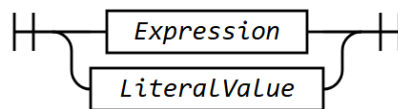
LiteralValue



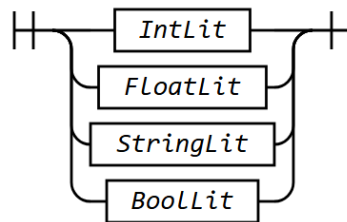
ElementTail



Element



Literal



BoolLit

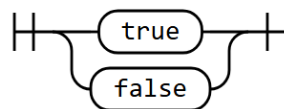
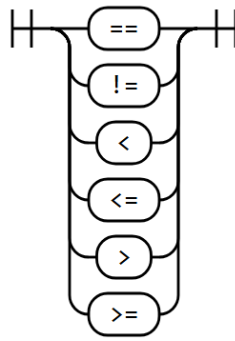


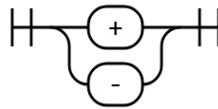
Рисунок 3.12 – Синтаксичні діаграми літералів у виразах

RelOp	=	'=='		'!='		'<'		'<='		'>'		'>='
AddOp	=	'+'		'-'								
MulOp	=	'*'		'/'		'%'						
UnaryOp	=	'-'		'+'		'!'						

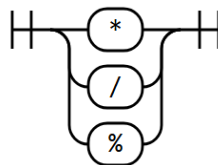
RelOp



AddOp



MulOp



UnaryOp

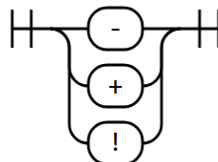


Рисунок 3.13 – Синтаксичні діаграми груп операторів

Приклади виразів:

```
-b + 4.5 * c           // (-b) + (4.5 * c)
2 ** 3 ** 2           // 2 ** (3 ** 2) = 512
-2 ** 2               // -(2 ** 2) = -4
x >= 0 && !done || n == 0
float(total) / float(len(v))
[3]int{1, 2, 3}
m[i][j] + p.x * (*ptr)
```

3.11 Властивості граматки

Властивості граматки перевірено автоматично програмою `tools/check_grammar.py`, яка обчислює множини FIRST і FOLLOW, буде

граф лівих виведень і перевіряє умови LL(1) для кожної альтернативи, необов'язкової частини та повторення. Результати перевірки наведено в табл. 3.1. Правильність перевірки підтверджено на навмисно зіпсованих варіантах граматики (з лівою рекурсією, конфліктом FIRST/FIRST, «висячим else» та недосяжним правилом): у кожному випадку дефект виявлено.

Таблиця 3.1 – Результати автоматичної перевірки граматики

№	Властивість граматики	Результат
1	Усі нетермінали, що використовуються, визначені	виконується
2	Грамматика контекстно-вільна: у лівій частині кожного правила один нетермінал	виконується
3	Недосяжні символи відсутні	виконується
4	Непродуктивні символи відсутні	виконується
5	Ліва рекурсія (безпосередня та опосередкована) відсутня	виконується
6	Синтаксична грамматика належить до класу LL(1), отже однозначна	виконується
7	Усі термінали складені із символів алфавіту	виконується
8	Синтаксична грамматика оперує токенами, а не символами	виконується

Коментарі до пунктів контрольного списку:

- **Контекстна вільність.** Ліва частина кожного з правил містить рівно один нетермінал.
- **Марні символи.** Кожен нетермінал досяжний від однієї з аксіом (Program, SourceText або Alphabet) і виводить термінальний ланцюжок.
- **Ліва рекурсія.** Відсутня: ліва асоціативність бінарних операторів виражена повтореннями { ... }, списки — правилами виду $X \{ ' , ' X \}$ або правою рекурсією (StatementTail, FieldTail, ElementTail).
- **Однозначність.** Кожна LL(1)-грамматика однозначна. Проблему «висячого else» усунено обов'язковими фігурними дужками, неоднозначність складених літералів у заголовках if/for (відома в мові Go) — вимогою явного типу [N]T у літералі масиву.
- **Алфавіт.** Усі термінали складаються із символів, визначених правилами Letter, Digit, SpecialChar, WhiteSpace та EndOfLine.

- **Розмежування граматики.** Синтаксична граматика звертається до лексичної лише через токени `Ident`, `IntLit`, `FloatLit`, `StringLit` та групи tokenів `BoolLit`, `RelOp`, `AssignOp`, `IncDecOp`.
- **Літерали.** Граматика охоплює всі форми цілих (без ведучих нулів), дійсних (з дробовою частиною, з експонентою, з пропущеною цілою або дробовою частиною), рядкових (порожній рядок, `escape`-послідовності) і логічних літералів; знак числа — унарний оператор.
- **Пріоритет, асоціативність і арність.** Пріоритет забезпечено стратифікацією, асоціативність — повторенням (ліва) та правою рекурсією (права, `**`). Унарні оператори (`UnaryOp`, `Dereference`, `AddrExpr`) і бінарні (`AddOp`, `MulOp`, `RelOp`, `&&`, `||`, `**`) описано окремими правилами.
- **Еквівалентність EBNF і діаграм.** Діаграми згенеровано з того самого файлу граматики; програма побудови документа перевіряє, що кожне правило подано в розд. 2–3 рівно один раз.

Граматикою виражено лише структуру програми. Контекстні умови перевіряються семантичним аналізатором; розподіл наведено в табл. 3.2.

Таблиця 3.2 – Інформація, виражена граматикою, та контекстні умови

Виражено граматикою	Контекстні умови (семантичний аналіз)
пріоритет і асоціативність операторів	сумісність типів операндів і результатів
обов'язкові фігурні дужки блоків	оголошеність імен і відсутність повторних оголошень
<code>struct</code> лише в оголошеннях типів	ліва частина <code>=</code> є l-значенням, ліва частина <code>:=</code> — ідентифікатором
<code>*T</code> лише в типах параметрів	інструкція-вираз є викликом функції
аргументи <code>Scan</code> мають форму <code>&вираз</code>	операнд <code>&</code> є адресованим виразом
константа має рівно одне ім'я та ініціалізатор	ініціалізатор константи є константним виразом
розіменування лише ідентифікатора	розіменовуваний ідентифікатор є параметром-вказівником
трикомпонентний <code>for</code> має всі три частини	умови <code>if/for</code> мають тип <code>bool</code> ; <code>post</code> -частина не є <code>:=</code>
оператори порівняння неасоціативні	довжина масиву — додатна константа типу <code>int</code>
типи та функції лише на верхньому рівні	наявність <code>return</code> у всіх гілках, сигнатура <code>main</code> , <code>import "fmt"</code>

4 Семантика та система типів

4.1 Модель виконання

Програма виконується в такому порядку:

1. Обчислюються значення глобальних констант і ініціалізуються глобальні змінні в порядку їх оголошення в тексті програми. Глобальні змінні без ініціалізатора отримують нульове значення свого типу.
2. Викликається функція `main`.
3. Після повернення з `main` програма завершується з кодом завершення 0.

Якщо під час виконання виникає помилка (розд. 4.11.2), програма негайно завершується з кодом 2, а повідомлення про помилку записується в стандартний потік помилок. Дані, виведені до помилки, зберігаються.

4.2 Типи даних

Кожне значення, змінна та вираз мовою Krok мають тип, відомий на етапі трансляції (статична типізація). Типи поділяються на скалярні (`int`, `float`, `bool`, `string` та користувацькі скалярні типи) і складені (масиви та структури). Вказівники існують лише як типи параметрів.

4.2.1 Вбудовані скалярні типи

Таблиця 4.1 – Вбудовані скалярні типи

Тип	Множина значень	Нульове значення	Літерали
<code>int</code>	цілі числа від –9223372036854775808 до 9223372036854775807 (64-бітне доповняльне представлення)	0	<code>IntLit</code>
<code>float</code>	числа з рухомою комою подвійної точності IEEE 754 (<code>binary64</code>), крім нескінченностей і NaN	0.0	<code>FloatLit</code>
<code>bool</code>	<code>false</code> , <code>true</code>	<code>false</code>	<code>true</code> , <code>false</code>

Тип	Множина значень	Нульове значення	Літерали
string	незмінні послідовності символів Unicode довільної довжини	""	StringLit

Мінімальне значення типу `int` не можна записати одним літералом; його отримують виразом `-9223372036854775807 - 1`.

4.2.2 Користувацькі скалярні типи

Оголошення `type T U`, де `U` — вбудований скалярний тип або інший користувацький скалярний тип, створює новий **іменований тип** `T` з тією самою множиною значень і операцій, що й у **базового типу** `U` (для ланцюжка оголошень базовим є вбудований тип у кінці ланцюжка).

Іменований тип відмінний від свого базового типу та від будь-якого іншого типу (номінальна еквівалентність): змінній типу `Celsius` не можна присвоїти значення типу `float` без явного приведення `Celsius(x)`. Виняток — літерали: числовий, рядковий або логічний літерал (зокрема зі знаком `+` чи `-`) неявно набуває іменованого типу, якщо його тип збігається з базовим типом або може бути до нього розширений (розд. 4.5.4).

```
type Celsius float
type Fahrenheit float
var t Celsius = 36.6           // літерал набуває типу Celsius
var f Fahrenheit = Fahrenheit(t*9/5 + 32)
// t = f                       // CE-05: різні іменовані типи
```

Бінарна арифметична операція над двома операндами одного іменованого типу дає результат того самого типу.

4.2.3 Масиви

Тип масиву `[N]T` описує послідовність рівно `N` елементів типу `T`, де `N` — константний вираз типу `int` зі значенням більше нуля (розд. 4.5.9), а `T` — будь-який тип. Багатовимірний масив — масив масивів: `[3][4]int` містить 3 елементи типу `[4]int`.

- **Ініціалізація.** Змінна-масив без ініціалізатора отримує масив нульових значень елементів. Статична ініціалізація виконується

літералом масиву $[N]T\{e_1, \dots, e_k\}$, де $k \leq N$; елементи з індексами від k до $N-1$ отримують нульові значення. Для масиву масивів внутрішні списки записуються без типу: $[2][2]\text{int}\{\{1, 2\}, \{3, 4\}\}$. Кількість елементів $k > N$ є помилкою SE-11. Динамічна ініціалізація виконується присвоєнням значень елементам під час виконання (наприклад, у циклі).

- **Індексування.** Вираз $a[i]$ позначає елемент масиву a з індексом i , де i — вираз типу `int`. Елементи нумеруються від 0 до $N-1$. Якщо значення індексу виходить за ці межі, виникає помилка часу виконання RE-04. Якщо індекс є константним виразом, вихід за межі виявляється на етапі трансляції (SE-11).
- **Довжина.** Вираз `len(a)` має тип `int` і дорівнює N ; для масиву він є константним виразом.
- **Семантика значень.** Масиви є типами-значеннями: присвоєння, ініціалізація, передача аргументу та повернення результату копіюють усі елементи.
- **Еквівалентність.** Типи масивів структурно еквівалентні: $[N]T_1$ і $[M]T_2$ ідентичні тоді й лише тоді, коли $N = M$ і типи T_1, T_2 ідентичні. Іменованій тип масиву (`type Row [3]int`) ідентичний лише самому собі, але значення неіменованого типу масиву з ідентичною структурою може бути йому присвоєне.
- Масиви не порівнюються операторами `==`, `!=` і не виводяться операцією `fmt.Print` безпосередньо (SE-05).

4.2.4 Структури

Оголошення `type T struct { ... }` створює структурний тип T із набором іменованих полів. Імена полів однієї структури мають бути унікальними (SE-12). Тип поля може бути будь-яким типом, крім самої структури T безпосередньо або через масиви чи інші структури, оскільки розмір такого значення був би нескінченним (SE-12).

- **Доступ до полів.** Вираз `s.f` позначає поле `f` значення `s` структурного типу. Звернення до неіснуючого поля є помилкою CE-12. Доступ до поля вкладеної структури записується ланцюжком: `seg.from.x`.
- **Ініціалізація.** Змінна структурного типу отримує значення, в якому кожне поле має нульове значення свого типу. Літералів структур мови не має; поля заповнюються присвоєнням.
- **Семантика значень.** Присвоєння структури копіює всі поля (зокрема вкладені масиви і структури) поелементно.
- **Еквівалентність.** Номінальна: кожне оголошення `struct` створює окремий тип, навіть якщо набори полів збігаються.
- Структури не порівнюються операторами `==`, `!=` і не виводяться безпосередньо (CE-05).

4.2.5 Параметри-вказівники

Тип `*T` («вказівник на `T`») може бути лише типом параметра функції. Він реалізує передачу параметра за **посиланням**.

- Аргументом для параметра типу `*T` є вираз `&x`, де `x` — адресований вираз (розд. 4.5.10) типу, ідентичного `T`, або інший параметр типу `*T`. Неявне розширення `int` $\rightarrow$ `float` для вказівників не застосовується.
- Вираз `*p`, де `p` — параметр типу `*T`, є l-значенням типу `T`, що позначає ту саму змінну, адресу якої передано у виклику. Суфікси застосовуються до розіменованого значення: `*p[i]` означає `(*p)[i]`, `*p.x` означає `(*p).x`.
- Параметру-вказівнику не можна присвоїти нове значення; вказівники не порівнюються, не виводяться і не беруть участі в арифметиці.
- Значення «порожнього вказівника» (`nil`) у мові немає: кожен параметр-вказівник завжди посилається на існуючу змінну. Оскільки вказівник не можна зберегти в змінній, полі чи результаті функції, він не може пережити змінну, на яку посилається.

4.2.6 Еквівалентність та сумісність типів

Два типи **ідентичні**, якщо це той самий вбудований тип, той самий іменований (користувацький скалярний, іменований масивний або структурний) тип, або якщо обидва є неіменованими типами масивів з однаковою довжиною та ідентичними типами елементів.

Значення x типу V **сумісне за присвоєнням** із типом T (може бути присвоєне змінній типу T , передане параметру типу T , повернене як результат типу T або використане як елемент масиву з елементами типу T), якщо виконується одна з умов:

1. V і T ідентичні;
2. V — `int`, T — `float` (неявне розширення, розд. 4.5.4);
3. x — літерал (можливо, з унарним `+` чи `-`), а T — іменований скалярний тип, до базового типу якого літерал сумісний за умовами 1–2;
4. V — неіменований тип масиву, T — іменований тип масиву з ідентичною структурою.

4.3 Оголошення, змінні та константи

Обов'язковість оголошення. Кожен ідентифікатор, крім імені пакета `main`, має бути оголошений до використання в межах своєї області видимості (розд. 4.4). Звернення до неоголошеного імені є помилкою CE-03. Повторне оголошення імені в тому самому блоці (зокрема імені параметра на верхньому рівні тіла функції) є помилкою CE-04.

Оголошення змінних. Оголошення `var` має три форми:

- `var a, b T` — змінні типу T з нульовими значеннями;
- `var a, b T = e1, e2` — значення виразів мають бути сумісними за присвоєнням з типом T ;
- `var a, b = e1, e2` — тип кожної змінної збігається з типом відповідного виразу.

Кількість виразів має дорівнювати кількості імен (CE-05). Вирази обчислюються зліва направо до того, як імена стають видимими.

Оголошення з ініціалізацією. Інструкція `x := e` оголошує в поточному блоці нову змінну `x` з типом виразу `e` і присвоює їй значення `e`. Ліва частина має бути ідентифікатором, не оголошеним у поточному блоці (CE-04, CE-17). Вираз `e` не може бути викликом функції без результату (CE-15). Оголошення `:=` дозволене лише всередині функцій.

Неініціалізовані змінні. Змінна, оголошена без ініціалізатора, отримує нульове значення свого типу (табл. 4.1); для масивів і структур — поелементно. Отже, читання змінної до першого явного присвоєння не є помилкою.

Глобальні змінні. Ініціалізатори глобальних змінних мають бути константними виразами (CE-16).

Іменовані константи. Оголошення `const C T = e` або `const C = e` вводить ім'я `C` для значення константного виразу `e`, обчисленого на етапі трансляції. Тип константи — `T` (значення `e` має бути з ним сумісним) або тип виразу `e`. Константи можуть мати лише скалярні типи. Константа не є змінною: її не можна змінити присвоєнням, `++/--` чи введенням, і до неї не можна застосувати оператор `&` (CE-07).

4.4 Області видимості

Мова Krok використовує статичні (лексичні) області видимості. **Блок** — ділянка тексту програми, у якій оголошені імена видимі. Блоки утворюють ієрархію:

1. **Блок пакета** охоплює весь файл. У ньому оголошуються глобальні константи, змінні, типи та функції. Типи й функції видимі в усьому файлі незалежно від порядку оголошень, що дозволяє взаємну рекурсію. Глобальні константи і змінні видимі в тілах усіх функцій, а в ініціалізаторах глобальних оголошень і довжинах масивів — лише після свого оголошення.
2. **Блок функції** містить параметри та інструкції верхнього рівня тіла функції.

3. **Неявний блок інструкції for** містить змінні, оголошені в її ініціалізаційній частині; він охоплює заголовок і тіло циклу.

4. **Явні блоки:** тіла гілок if/else, тіла циклів і вкладені блоки { ... }.

Область видимості локальної змінної чи константи починається одразу після її оголошення (після ініціалізатора) і закінчується в кінці найглибшого блоку, що містить оголошення. Тому у виразі `x := x + 1` ім'я `x` праворуч позначає змінну зовнішнього блоку.

Затінення. Ім'я, оголошене у вкладеному блоці, затінює однойменне ім'я зовнішнього блоку (зокрема глобальне) до кінця вкладеного блоку. Під час розв'язання імені пошук іде від найглибшого блоку до блоку пакета; використовується перше знайдене оголошення.

```
var x int = 1          // глобальна x
func main() {
    x := "local"       // затінює глобальну x
    {
        x := 3.5       // затінює локальну x
        fmt.Println(x) // 3.5
    }
    fmt.Println(x)     // local
}
```

Час життя. Глобальні змінні існують протягом усього виконання програми. Локальні змінні та параметри існують від входу до блоку (виклику функції) до виходу з нього; при кожному новому виконанні блоку створюються нові екземпляри його змінних.

4.5 Вирази

4.5.1 Пріоритет та асоціативність операторів

Таблиця 4.2 – Пріоритет та асоціативність операторів

Пріоритет	Оператори	Опис	Асоціативність
8 (найвищий)	.f, [i], (args), *p	доступ до поля, індексування, виклик, розіменування	ліва
7	**	піднесення до степеня	права
6	унарні -, +, !	зміна знака, тотожність, заперечення	права (префіксні)

Пріоритет	Оператори	Опис	Асоціативність
5	*, /, %	мультиплікативні	ліва
4	+, -	адитивні, конкатенація рядків	ліва
3	==, !=, <, <=, >, >=	порівняння	неасоціативні
2	&&	кон'юнкція	ліва
1 (найнижчий)		диз'юнкція	ліва

Оператор ****** має вищий пріоритет, ніж унарні оператори зліва від нього, але його правий операнд може починатися з унарного оператора: `-2 ** 2` дорівнює `-(2 ** 2) = -4`, а `2.0 ** -1` дорівнює `0.5`. Оператор **&** (взяття адреси) не є оператором виразу: він може стояти лише перед аргументом виклику.

4.5.2 Порядок обчислення

Операнди бінарних операторів, аргументи викликів, індекси та елементи літералів масивів обчислюються **зліва направо**. Оператори **&&** та **||** обчислюються за скороченою схемою: правий операнд **&&** обчислюється лише тоді, коли лівий дорівнює `true`, а правий операнд **||** — лише тоді, коли лівий дорівнює `false`.

4.5.3 Арифметичні оператори

Таблиця 4.3 – Типи операндів і результатів арифметичних операторів

Оператор	Операція	Типи операндів	Тип результату
+	додавання	int, int	int
-, *	віднімання, множення	int, int	int
+, -, *	те саме	float i float або int	float
/	ділення	int, int	int (частка, округлена в бік нуля)
/	ділення	float i float або int	float
%	остача	int, int	int
**	ступінь	int, int	int
**	ступінь	float i float або int	float
+	конкатенація	string, string	string
унарні +, -	тотожність, зміна знака	int або float	тип операнда

Для іменованих типів діють ті самі правила відносно базового типу за умови, що обидва операнди мають той самий іменований тип (або один із них — літерал).

- Для цілих a / b і $a \% b$ виконується тотожність $a = (a / b) * b + a \% b$; знак остачі збігається зі знаком a : $-7 / 2 = -3$, $-7 \% 2 = -1$.
- Цілочислове піднесення до степеня $a ** b$ вимагає $b \geq 0$ (інакше RE-05); $0 ** 0 = 1$. Дійсне піднесення до степеня виконується за правилами функції `pow` стандарту IEEE 754.
- Ділення на нуль (цілочислове чи дійсне) та взяття остачі від ділення на нуль спричиняють RE-01.
- Якщо результат цілочислової операції (зокрема унарного мінуса, `++`, `--` та `**`) не належить множині значень `int`, виникає RE-02. Якщо результат дійсної операції є нескінченністю або NaN, виникає RE-03; втрата точності та антипереповнення (округлення до нуля) помилками не є.
- Оператори `%` для `float`, а також будь-які арифметичні оператори для `bool`, масивів і структур недопустимі (SE-05).

4.5.4 Неявне приведення типів

Єдине неявне приведення в мові — **розширення** значення типу `int` до типу `float`: ціле число інтерпретується як дійсне з нульовою дробовою частиною, округлене до найближчого значення `float`, якщо воно не представляється точно. Розширення виконується:

- в арифметичній операції та операції порівняння, якщо один операнд має тип `int`, а інший — `float` (перетворюється операнд типу `int`);
- у присвоєнні та ініціалізації змінної типу `float` значенням типу `int`;
- під час передачі аргументу типу `int` параметру типу `float` та повернення значення типу `int` з функції з результатом типу `float`;
- для елемента типу `int` у літералі масиву з елементами типу `float`.

Звуження `float` → `int` неявно не виконується ніколи: `var n int = 2.5` є помилкою CE-05. Неявні перетворення між `bool`, `string` і числовими типами відсутні.

4.5.5 Явне приведення типів

Явне приведення записується як `T(x)`, де `T` — вбудований або користувацький скалярний тип (для користувацького типу — синтаксично виклик, який семантичний аналізатор розпізнає за тим, що ідентифікатор позначає тип).

Таблиця 4.4 – Допустимі явні приведення

Тип <code>x</code>	Тип <code>T</code>	Результат
числовий	<code>float</code>	значення <code>x</code> як дійсне число
<code>float</code>	<code>int</code>	значення <code>x</code> з відкинутою дробовою частиною (округлення в бік нуля): <code>int(2.9) = 2</code> , <code>int(-2.9) = -2</code> ; якщо результат не належить множині значень <code>int</code> — RE-09
<code>int</code>	<code>int</code>	значення без змін
тип з базовим <code>U</code>	тип з базовим <code>U'</code>	приведення <code>U</code> → <code>U'</code> за рядками вище, якщо обидва базові типи числові або однакові

Приведення між `bool`, `string` та числовими типами, а також приведення масивів і структур недопустимі (CE-06).

4.5.6 Оператори порівняння

Результат порівняння завжди має тип `bool`.

- Числові операнди (`int`, `float`, у разі змішаних типів — з розширенням до `float`) порівнюються за значенням усіма шістьма операторами.
- Рядки порівнюються всіма шістьма операторами лексикографічно за кодами символів Unicode.
- Логічні значення порівнюються лише операторами `==` та `!=`.

- Операнди іменованих типів порівнюються, якщо мають той самий тип (або один із них — літерал), за правилами базового типу.
- Масиви, структури та вказівники не порівнюються (CE-05).

4.5.7 Логічні оператори

Оператори `&&`, `||` та `!` застосовуються лише до операндів типу `bool` (або іменованого типу з базовим `bool`) і дають результат того самого типу. Семантику скороченого обчислення наведено в розд. 4.5.2. Числа та рядки не інтерпретуються як логічні значення: `if n { }` для `n` типу `int` є помилкою CE-05.

4.5.8 Операції над рядками

Для рядків визначено конкатенацію `+`, порівняння (розд. 4.5.6) та функцію `len(s)`, яка повертає кількість символів Unicode в рядку (значення типу `int`). Рядки незмінні: індексування рядків не підтримується.

4.5.9 Константні вирази

Константний вираз обчислюється на етапі трансляції. Він може містити лише літерали, імена констант, оператори (крім викликів функцій та `&`), явні приведення вбудованих типів і вирази `len(a)` для масивів. Помилки обчислення константного виразу (ділення на нуль, переповнення, від'ємний цілий показник степеня) є помилками трансляції CE-10.

4.5.10 L-значення

L-значення (адресований вираз) позначає місце зберігання значення і може стояти ліворуч від `=`, бути операндом `++/--`, операндом `&` чи аргументом `fmt.Scan`. L-значеннями є:

- ім'я змінної або параметра, що не є вказівником;
- розіменування параметра-вказівника `*p`;
- індексування `a[i]`, якщо `a` є l-значенням;
- доступ до поля `s.f`, якщо `s` є l-значенням.

Не є l-значеннями: константи, літерали, результати викликів функцій і приведень, вирази з операторами, імена функцій і типів, параметри-вказівники без розіменування.

4.6 Інструкції

4.6.1 Присвоєння

Інструкція $v = e$ обчислює вирази-індекси в лівій частині (зліва направо), потім вираз e , після чого записує значення e у місце, позначене l-значенням v . Ліва частина має бути l-значенням (СЕ-07), а значення e — сумісним за присвоєнням з типом v (СЕ-05). Для масивів і структур присвоєння копіює значення повністю.

Інструкції $v++$ і $v--$ еквівалентні $v = v + 1$ і $v = v - 1$ з однократним обчисленням v . Операнд має бути l-значенням числового типу.

Простою інструкцією, що складається лише з виразу, може бути тільки виклик функції (СЕ-17); результат функції в такому разі відкидається.

4.6.2 Блок

Інструкції блоку виконуються послідовно в порядку запису. Блок утворює нову область видимості (розд. 4.4).

4.6.3 Розгалуження

Під час виконання інструкції `if E B1 else B2` обчислюється умова E , яка має тип `bool` (СЕ-05). Якщо значення умови `true`, виконується блок $B1$, інакше — гілка `else` (блок або вкладена інструкція `if`), якщо вона є. Ланцюжок `if ... else if ... else` перевіряє умови по черзі і виконує рівно одну гілку: першу, умова якої істинна, або гілку `else`.

4.6.4 Цикли

Цикл із передумовою `for E B` виконується так:

1. Обчислюється умова E типу `bool` (СЕ-05).

2. Якщо значення `false`, виконання інструкції завершується.
 3. Виконується тіло `B`, після чого керування переходить до кроку 1.
- Тіло може не виконатися жодного разу.

Трикомпонентний цикл `for S1; E; S2` `B` створює неявний блок і виконується так:

1. Один раз виконується ініціалізаційна інструкція `S1` (присвоєння, оголошення `:=`, `++/--` або виклик). Змінні, оголошені в `S1`, видимі в `E`, `S2` та `B`.
2. Обчислюється умова `E` типу `bool`; якщо значення `false`, виконання інструкції завершується.
3. Виконується тіло `B`.
4. Виконується інструкція наступного кроку `S2` (присвоєння, `++/--` або виклик; оголошення `:=` недопустиме — СЕ-17), після чого керування переходить до кроку 2.

Змінна циклу — звичайна змінна: її значення можна змінювати в тілі циклу, і ці зміни впливають на подальші ітерації. Кількість ітерацій заздалегідь не обчислюється. Після завершення циклу змінні з `S1` недоступні.

```
for k := 10; k > 0; k-- {  
    k = k - 2           // дозволено: виведе 8 5 2 -1  
    fmt.Print(k, " ")  
}
```

4.6.5 Повернення з функції

Інструкція `return` завершує виконання поточної функції. У функції без результату вона записується без виразу (СЕ-09); у функції з результатом — з виразом, сумісним за присвоєнням з типом результату, і це значення стає значенням виклику.

Функція з результатом повинна завершуватися інструкцією `return` на кожному шляху виконання (СЕ-09). Це перевіряється статично: список інструкцій вважається **завершальним**, якщо його остання інструкція є `return`, завершальним блоком або інструкцією `if` з гілкою `else`, у якій усі гілки

завершальні. Цикли завершальними не вважаються. Функція без результату повертає керування після виконання останньої інструкції тіла.

4.7 Функції

Оголошення. Функції оголошуються лише на верхньому рівні; вкладені функції не підтримуються. Ім'я функції унікальне серед імен блоку пакета; перевантаження функцій немає. Функції не є значеннями: їх можна лише викликати.

Функція `main`. Програма повинна містити функцію `main` без параметрів і без результату (CE-13). Виконання програми починається з виклику `main`.

Виклик. Під час виклику $f(a_1, \dots, a_n)$:

1. Аргументи обчислюються зліва направо. Кількість аргументів має дорівнювати кількості параметрів, а кожен аргумент — бути сумісним за присвоєнням із типом відповідного параметра (CE-08).
2. Створюється новий фрейм активації в стеку викликів, який містить параметри, локальні змінні та адресу повернення.
3. Параметрам за значенням (тип `T`) присвоюються копії значень аргументів; для масивів і структур копіюються всі елементи, тому зміна параметра не впливає на аргумент. Параметрам за посиланням (тип `*T`) передаються адреси змінних (розд. 4.2.5); присвоєння `*p` змінює саму змінну викликача.
4. Виконується тіло функції до інструкції `return` або до кінця тіла.
5. Фрейм знищується, керування повертається в точку виклику; значенням виклику є значення з `return`.

Виклик функції без результату може бути лише інструкцією (CE-15).

Рекурсія. Функція може викликати себе безпосередньо або опосередковано. Кожен рекурсивний виклик створює власний фрейм з окремими екземплярами параметрів і локальних змінних. Максимальна глибина стека викликів визначається реалізацією, але має бути не меншою за 10 000 вкладених викликів; її перевищення спричиняє RE-06.

4.8 Введення та виведення

Операції введення/виведення доступні після оголошення `import "fmt";` їх використання без імпорту є помилкою CE-14.

Виведення. `fmt.Print(e1, ..., en)` записує текстові представлення значень у стандартний потік виведення без роздільників і без переходу на новий рядок. `fmt.Println(e1, ..., en)` записує представлення, розділені одним пробілом, і завершує виведення символом нового рядка; `fmt.Println()` виводить лише символ нового рядка. Аргументи мають бути скалярних типів (CE-05). Текстові представлення значень:

- `int` — десятковий запис, від'ємні числа — зі знаком -: 42, -7;
- `float` — найкоротший десятковий запис, який однозначно відновлює значення; якщо $1e-4 \leq |x| < 1e16$, запис має звичайну форму і завжди містить крапку та принаймні одну цифру після неї (5.0, 0.125, 43.6), інакше — експоненційну форму ($1e+16$, $2.5e-05$);
- `bool` — `true` або `false`;
- `string` — символи рядка без лапок; escape-послідовності виводяться як відповідні символи;
- значення іменованого типу — за правилом базового типу.

Введення. `fmt.Scan(&v1, ..., &vn)` послідовно для кожного аргументу пропускає пробільні символи (пробіл, табуляцію, кінець рядка) у стандартному потоці введення, читає слово — найдовшу послідовність непробільних символів — і перетворює його відповідно до типу змінної:

- `int` — необов'язковий знак і десяткові цифри (ведучі нулі допускаються); значення має належати множині значень `int`;
- `float` — необов'язковий знак і запис цілого або дійсного числа за формою `IntLit/FloatLit` (з допуском ведучих нулів);
- `bool` — слово `true` або `false`;
- `string` — слово без змін (рядок, що містить пробіли, одним викликом прочитати не можна);
- іменований тип — за правилом базового типу.

Прочитане значення одразу присвоюється змінній. Змінні масивних і структурних типів як цілі не вводяться (CE-05), але їхні скалярні елементи та поля — вводяться. Якщо слово не можна перетворити на значення потрібного типу, виникає RE-07; якщо потік введення закінчився раніше, ніж прочитано всі значення, — RE-08.

Перед кожною операцією введення та перед завершенням програми буфер виведення примусово записується в потік, тому запрошення до введення з'являються до очікування даних.

4.9 Модульність

Програма мовою Krok складається з одного файлу, що містить єдиний пакет `main` (ім'я пакета, відмінне від `main`, — CE-13). Розділення програми на кілька файлів чи користувацьких модулів не підтримується.

Оголошення імпорту підключає вбудований пакет. Єдиний доступний пакет — `"fmt"`; імпорт іншого шляху або повторний імпорт `"fmt"` є помилкою CE-14, отже циклічний імпорт неможливий. Доступ до операцій пакета завжди кваліфікований: `fmt.Println`. Ім'я `fmt` не входить до жодного блоку видимості і не може бути затінене, оскільки є ключовим словом.

4.10 Керування пам'яттю

Мова Krok не має динамічного виділення пам'яті та збирання сміття. Пам'ять розподіляється так:

- **статична область** — глобальні змінні; створюється перед викликом `main` і існує до завершення програми;
- **стек викликів** — фрейми активації функцій з параметрами та локальними змінними, зокрема масивами і структурами, розмір яких відомий на етапі трансляції; фрейм створюється під час виклику і знищується під час повернення;
- константи не займають пам'яті під час виконання як змінні: їхні значення підставляються транслятором.

Рядки незмінні, тому реалізація може розміщувати їхній вміст у керованій нею пам'яті та спільно використовувати однакові значення; це не спостерігається програмою. Висячі посилання неможливі, оскільки вказівники існують лише як параметри і не можуть пережити змінну, на яку посилаються (розд. 4.2.5).

4.11 Помилки

4.11.1 Помилки трансляції

Транслятор повинен виявити всі наведені нижче помилки до початку виконання програми; програма з помилкою трансляції не виконується.

Таблиця 4.5 – Помилки трансляції

Код	Помилка
CE-01	лексична помилка: недопустимий символ, ведучий нуль у цілому літералі, неправильна експонента, незакритий рядковий літерал, невідома escape-послідовність
CE-02	синтаксична помилка: послідовність токенів не відповідає граматиці
CE-03	використання неоголошеного імені
CE-04	повторне оголошення імені в тому самому блоці
CE-05	невідповідність типів: операнди оператора, присвоєння, ініціалізація, умова if/for, аргументи Print/Scan, кількість виразів у var
CE-06	недопустиме явне приведення типів
CE-07	присвоєння, ++/--, & чи введення для виразу, що не є l-значенням (константи, результату виклику тощо)
CE-08	невідповідність кількості або типів аргументів виклику; виклик імені, що не є функцією чи типом
CE-09	return без виразу у функції з результатом, з виразом — у функції без результату; відсутність return на деякому шляху виконання
CE-10	значення літерала поза множиною значень типу; помилка обчислення константного виразу
CE-11	довжина масиву не є додатною константою типу int; зайві елементи в літералі масиву; константний індекс поза межами

Код	Помилка
CE-12	повторне або неіснуюче поле структури; рекурсивне включення структури в себе
CE-13	ім'я пакета відмінне від main; відсутність функції main або неправильна її сигнатура
CE-14	використання fmt без імпорту; імпорт шляху, відмінного від "fmt"; повторний імпорт
CE-15	використання виклику функції без результату як значення
CE-16	ініціалізатор глобальної змінної не є константним виразом
CE-17	ліва частина := не є ідентифікатором; := в інструкції наступного кроку for; інструкція-вираз не є викликом функції

4.11.2 Помилки часу виконання

У разі помилки часу виконання програма припиняє роботу з кодом завершення 2 і записує в стандартний потік помилок повідомлення у форматі `runtime error RE-NN at рядок:стовпчик: опис`.

Таблиця 4.6 – Помилки часу виконання

Код	Помилка	Причина
RE-01	ділення на нуль	$a / 0$, $a \% 0$ для <code>int</code> ; $x / 0.0$ для <code>float</code>
RE-02	переповнення цілого	результат <code>+</code> <code>-</code> <code>*</code> <code>**</code> <code>++</code> <code>--</code> чи унарного <code>-</code> поза межами <code>int</code> (зокрема <code>-9223372036854775807 - 2</code>)
RE-03	переповнення дійсного	результат дійсної операції — нескінченність або NaN (<code>1e308 * 10</code> , <code>(-8.0) ** 0.5</code>)
RE-04	вихід за межі масиву	індекс $i < 0$ або $i \geq \text{len}(a)$
RE-05	від'ємний показник	$a ** b$ для <code>int</code> з $b < 0$
RE-06	переповнення стека	перевищено максимальну глибину викликів (нескінченна рекурсія)
RE-07	некоректні вхідні дані	слово, прочитане <code>fmt.Scan</code> , не є значенням типу змінної
RE-08	кінець вхідного потоку	потік введення закінчився під час виконання <code>fmt.Scan</code>
RE-09	недопустиме приведення	<code>int(x)</code> , де значення x після відкидання дробової частини поза межами <code>int</code>

5 Приклади програм

Усі наведені нижче програми (крім прикладів із розд. 5.10) є лексично і синтаксично правильними, що перевірено програмою `tools/krok_parse.py` за граматикою з розд. 6.

5.1 Базовий приклад

Базовий приклад містить принаймні один екземпляр кожної лексеми з таблиці токенів (табл. 2.2); повноту покриття перевірено автоматично командою

```
python tools/krok_parse.py --coverage examples/base_example.krok.
```

```
1 // Базовий приклад програми мовою Krok:
2 // містить приклади всіх лексем із таблиці токенів.
3 package main
4
5 import "fmt"
6
7 const Size int = 5 // іменована константа з явним типом
8 const Eps = 1.5e-3 // тип float визначається за літералом
9 const Title = "Krok:\t\"крок\" \\\ 2026"
10
11 type Celsius float // користувацький скалярний тип
12
13 type Point struct {
14     x, y float
15     label string
16 }
17
18 var total int // глобальна змінна, початкове значення 0
19 var ready bool = false // глобальна змінна з ініціалізацією
20
21 // Рекурсивне обчислення факторіала.
22 func factorial(n int) int {
23     if n <= 1 {
24         return 1
25     }
26     return n * factorial(n-1)
27 }
28
29 // Передача параметрів за посиланням.
30 func swap(a, b *int) {
31     t := *a
32     *a = *b
33     *b = t
34 }
35
36 // Масив передається за значенням (копіюється).
37 func sum(v [Size]int) int {
38     s := 0
39     for i := 0; i < len(v); i++ {
```

```

40     s = s + v[i]
41 }
42 return s
43 }
44
45 func distance(p Point) float {
46     return (p.x**2 + p.y**2) ** .5
47 }
48
49 func main() {
50     var n, m int
51     fmt.Scan(&n, &m)
52     swap(&n, &m)
53     fmt.Println("n =", n, "m =", m, "n! =", factorial(n%10))
54
55     var data [Size]int = [Size]int{3, -1, 4, 1, +5}
56     var grid [2][3]float
57     grid[1][2] = 2.
58     total = sum(data)
59
60     var t Celsius = 36.6
61     t = t + Celsius(n)
62     x := float(total)/2 - 7*3 // int неявно розширюється до float
63     k := int(-x)
64
65     var p Point
66     p.x = 3.0
67     p.y = 4E0
68     p.label = ""
69     d := distance(p)
70
71     ok := true
72     ready = d >= 5.0 && !(k == 0) || total != 12
73     for k > 10 {
74         k--
75     }
76     for i := 0; i < 3; i++ {
77         if grid[1][2] > float(i) {
78             fmt.Print(i, " ")
79         } else if i == 2 {
80             fmt.Print("last")
81         } else {
82             fmt.Print("-")
83         }
84     }
85     fmt.Println()
86     fmt.Print(Title, "\n")
87     fmt.Println(t, d, ready, ok, Eps, x < -1.0, 2**3**2, k)
88 }

```

Для вхідних даних 4 7 програма виводить:

```

n = 7 m = 4 n! = 5040
0 1 last
Krok: "крок" \ 2026
43.6 5.0 true true 0.0015 true 512 10

```

5.2 Типи та приведення

```
1 // Типи, неявне та явне приведення
2 package main
3
4 import "fmt"
5
6 func main() {
7     var i int = 7
8     var f float = 2 // int-літерал неявно розширюється до float
9     a := i / 2      // int / int → int: 3 (відкидання дробової частини)
10    b := i / 2.0     // int / float → float: 3.5
11    c := float(i) / 2 // явне приведення: 3.5
12    d := int(-3.9)   // відкидання дробової частини в бік нуля: -3
13    e := i % 4       // остача: 3
14    s := "Kr" + "ok" // конкатенація рядків
15    fmt.Println(a, b, c, d, e, s, f ** 2, len(s))
16 }
```

Результат виконання: 3 3.5 3.5 -3 3 Krok 4.0 4.

5.3 Розгалуження

```
1 // Розгалуження: if / else if / else
2 package main
3
4 import "fmt"
5
6 func main() {
7     var t float
8     fmt.Scan(&t)
9     if t < 0 {
10         fmt.Println("мороз")
11     } else if t < 15 {
12         fmt.Println("прохолодно")
13     } else if t <= 25 && t != 20 {
14         fmt.Println("комфортно")
15     } else {
16         fmt.Println("спекотно або рівно 20")
17     }
18 }
```

Для вхідного значення 22 програма виводить комфортно, для 20 — спекотно або рівно 20, для -3.5 — мороз.

5.4 Цикли

```
1 // Цикли: з передумовою та з лічильником (трикомпонентний)
2 package main
3
4 import "fmt"
5
6 func main() {
7     var n int
8     fmt.Scan(&n)
9 }
```

```

10 // цикл з передумовою: сума цифр числа
11 digits := 0
12 for n > 0 {
13     digits = digits + n%10
14     n = n / 10
15 }
16 fmt.Println("сума цифр:", digits)
17
18 // вкладені цикли з лічильником: таблиця множення 3x3
19 for i := 1; i <= 3; i++ {
20     for j := 1; j <= 3; j++ {
21         fmt.Print(i*j, "\t")
22     }
23     fmt.Println()
24 }
25
26 // лічильник можна змінювати в тілі циклу
27 for k := 10; k > 0; k-- {
28     k = k - 2
29     fmt.Print(k, " ")
30 }
31 }

```

Для вхідного значення 2026 програма виводить:

```

сума цифр: 10
1   2   3
2   4   6
3   6   9
8 5 2 -1

```

5.5 Функції та рекурсія

```

1 // Функції та рекурсія
2 package main
3
4 import "fmt"
5
6 func fib(n int) int {
7     if n < 2 {
8         return n
9     }
10    return fib(n-1) + fib(n-2)
11 }
12
13 func gcd(a, b int) int {
14     if b == 0 {
15         return a
16     }
17     return gcd(b, a%b)
18 }
19
20 func power(base float, e int) float {
21     if e == 0 {
22         return 1
23     } else if e < 0 {
24         return 1 / power(base, -e)
25     }

```

```

26     return base * power(base, e-1)
27 }
28
29 func main() {
30     fmt.Println(fib(10), gcd(84, 36), power(2, -3), 2.0**-3)
31 }

```

Результат виконання: 55 12 0.125 0.125.

5.6 Передача параметрів за посиланням

```

1 // Передача параметрів за посиланням: сортування масиву
2 package main
3
4 import "fmt"
5
6 const N = 6
7
8 func swap(a, b *int) {
9     t := *a
10    *a = *b
11    *b = t
12 }
13
14 func sort(a *[N]int) {
15     for i := 0; i < N-1; i++ {
16         for j := 0; j < N-1-i; j++ {
17             if (*a)[j] > (*a)[j+1] {
18                 swap(&(*a)[j], &(*a)[j+1])
19             }
20         }
21     }
22 }
23
24 func main() {
25     var v [N]int
26     for i := 0; i < N; i++ {
27         fmt.Scan(&v[i])
28     }
29     sort(&v)
30     fmt.Println(v[0], v[1], v[2], v[3], v[4], v[5])
31 }

```

Для вхідних даних 5 2 9 1 7 3 програма виводить 1 2 3 5 7 9.

5.7 Масиви та структури

```

1 // Масиви (статична та динамічна ініціалізація) і структури
2 package main
3
4 import "fmt"
5
6 type Vec struct {
7     x, y float
8 }
9
10 type Segment struct {

```

```

11     from, to Vec
12     name      string
13 }
14
15 func length(s Segment) float {
16     dx := s.to.x - s.from.x
17     dy := s.to.y - s.from.y
18     return (dx*dx + dy*dy) ** 0.5
19 }
20
21 func main() {
22     m := [2][2]int{
23         {1, 2},
24         {3, 4},
25     }
26     var sq [5]int // нульові значення
27     for i := 0; i < len(sq); i++ {
28         sq[i] = i * i // динамічна ініціалізація
29     }
30
31     var s Segment
32     s.name = "AB"
33     s.to.x = 3
34     s.to.y = 4
35     c := s // поелементне копіювання
36     c.name = "copy"
37
38     fmt.Println(m[1][0], sq[4], length(s), s.name, c.name)
39 }

```

Результат виконання: 3 16 5.0 AB copy.

5.8 Области видимості та затінення

```
1 // Области видимості та затінення
2 package main
3
4 import "fmt"
5
6 var x int = 1 // глобальна
7
8 func show() {
9     fmt.Println("global x =", x) // глобальна x: 1
10 }
11
12 func main() {
13     fmt.Println(x) // 1
14     x := "local"    // затінює глобальну x у тілі main
15     {
16         x := 3.5 // затінює локальну x у вкладеному блоці
17         fmt.Println(x)
18     }
19     fmt.Println(x) // local
20     for i := 0; i < 2; i++ {
21         x := i // нова змінна на кожне виконання тіла
22         fmt.Println(x)
23     }
24     show()
25 }
```

Результат виконання:

```
1
3.5
local
0
1
global x = 1
```

5.9 Помилки часу виконання

```
1 // Помилки часу виконання
2 package main
3
4 import "fmt"
5
6 func depth(n int) int {
7     return depth(n+1) + 1 // RE-06: переповнення стека
8 }
9
10 func main() {
11     var a [3]int
12     var i, z int
13     fmt.Scan(&i, &z)
14     fmt.Println(a[i]) // RE-04 при i < 0 або i > 2
15     fmt.Println(10 / z) // RE-01 при z == 0
16     big := 9223372036854775807
17     fmt.Println(big + 1) // RE-02: переповнення int
18     fmt.Println(depth(0))
19 }
```

Для вхідних даних 5 1 виконання припиняється на першому ж виведенні: runtime error RE-04 at 14:15: index 5 out of range [0, 2]. Для 1 0 програма виводить 0 і завершується з RE-01 у рядку 15.

5.10 Некоректні програми

Таблиця 5.1 – Приклади некоректних фрагментів програм

Фрагмент	Помилка
} i else { у різних рядках	CE-02: після } вставлено ;, очікується }, отримано else
if x > 0 x = 2	CE-02: очікується {
x := 007	CE-01: цілий літерал із ведучим нулем
type Local int у тілі функції	CE-02: оголошення типу лише на верхньому рівні
var p *int	CE-02: вказівник може бути лише типом параметра
a := 1 < 2 < 3	CE-02: оператори порівняння неасоціативні
a = b = 3	CE-02: присвоєння є інструкцією, а не виразом
fmt.Scan(n)	CE-02: аргумент Scan має форму &вираз
var n int = 2.5	CE-05: неявне звуження float → int заборонене
if n { } для n int	CE-05: умова має тип int, а не bool
const N = 3 i N = 4	CE-07: присвоєння константі
x := 1 i x := 2 в одному блоці	CE-04: повторне оголошення
func f() int { if a { return 1 } }	CE-09: немає return на шляху, коли умова хибна
var a [0]int	CE-11: довжина масиву має бути додатною
fmt.Println(int("5"))	CE-06: приведення string → int недопустиме

Перші вісім фрагментів відхиляються вже синтаксичним аналізатором; їх повні тексти містяться в каталозі examples/invalid і перевіряються командою python tools/krok_parse.py --invalid examples/invalid/*.krok.

6 Повна граматика мови

Повну граматика мови Krok наведено в порядку від кореневого нетермінала Program до низькорівневих лексичних конструкцій Letter та Digit. Текст збігається з файлом grammar/krok.ebnf, з якого згенеровано всі синтаксичні діаграми.

```
(* =====
Krok – повна граматика мови (РБНФ)
Синтаксична аксіома: Program
Лексична аксіома:   SourceText
Алфавіт:             Alphabet
===== *)

(* § Структура програми *)
Program      = PackageClause ';' { ImportDecl ';' } { TopLevelDecl ';' }
PackageClause = package Ident
ImportDecl   = import StringLit
TopLevelDecl = VarDecl | ConstDecl | TypeDecl | FuncDecl

(* § Оголошення *)
VarDecl      = var IdentList
              ( Type [ '=' ExpressionList ] | '=' ExpressionList )
ConstDecl    = const Ident [ Type ] '=' Expression
TypeDecl     = type Ident ( Type | StructType )
StructType   = struct '{' FieldList '}'
FieldList    = [ FieldDecl FieldTail ]
FieldTail    = [ ';' FieldList ]
FieldDecl    = IdentList Type
IdentList    = Ident { ',' Ident }

(* § Типи *)
Type         = BasicType | TypeName | ArrayType
BasicType    = int | float | bool | string
TypeName     = Ident
ArrayType    = '[' Expression ']' Type

(* § Функції *)
FuncDecl     = func Ident '(' [ ParamList ] ')' [ Type ] Block
ParamList    = ParamGroup { ',' ParamGroup }
ParamGroup   = IdentList ParamType
ParamType    = [ '*' ] Type

(* § Інструкції *)
Block        = '{' StatementList '}'
StatementList = [ Statement StatementTail ]
StatementTail = [ ';' StatementList ]
Statement    = VarDecl | ConstDecl | IfStmt | ForStmt | ReturnStmt
              | IoStmt | Block | SimpleStmt
SimpleStmt   = Expression [ AssignOp Expression | IncDecOp ]
AssignOp     = '=' | ':='
IncDecOp     = '++' | '--'
IfStmt       = if Expression Block [ else ( IfStmt | Block ) ]
ForStmt      = for ForHeader Block
ForHeader    = SimpleStmt [ ';' Expression ';' SimpleStmt ]
ReturnStmt   = return [ Expression ]
IoStmt       = fmt '.' ( OutputCall | InputCall )
```

```

OutputCall    = ( 'Print' | 'Println' ) '(' [ ExpressionList ] ')'
InputCall     = 'Scan' '(' AddrExpr { ',' AddrExpr } ')'

(* § Вирази *)
ExpressionList = Expression { ',' Expression }
Expression     = OrExpr
OrExpr         = AndExpr { '|' AndExpr }
AndExpr        = RelExpr { '&&' RelExpr }
RelExpr        = AddExpr [ RelOp AddExpr ]
AddExpr        = MulExpr { AddOp MulExpr }
MulExpr        = UnaryExpr { MulOp UnaryExpr }
UnaryExpr      = UnaryOp UnaryExpr | PowerExpr
PowerExpr      = PrimaryExpr [ '**' UnaryExpr ]
PrimaryExpr    = Operand { Selector | Index | Arguments }
Selector       = '.' Ident
Index          = '[' Expression ']'
Arguments      = '(' [ ArgumentList ] ')'
ArgumentList   = Argument { ',' Argument }
Argument       = AddrExpr | Expression
AddrExpr       = '&' PrimaryExpr
Operand        = Literal | Ident | Dereference | Conversion | LenCall
               | ArrayLit | '(' Expression ')'
Dereference    = '*' Ident
Conversion     = BasicType '(' Expression ')'
LenCall        = len '(' Expression ')'
ArrayLit       = ArrayType LiteralValue
LiteralValue   = '{' [ Element ElementTail ] '}'
ElementTail    = [ ',' [ Element ElementTail ] ]
Element        = Expression | LiteralValue
Literal        = IntLit | FloatLit | StringLit | BoolLit
BoolLit        = true | false
RelOp          = '==' | '!=' | '<' | '<=' | '>' | '>='
AddOp          = '+' | '-'
MulOp          = '*' | '/' | '%'
UnaryOp        = '-' | '+' | '!'

(* =====
   Лексична граматика
   ===== *)

(* § Лексеми *)
SourceText    = { Lexeme }
Lexeme        = Token | Comment | WhiteSpace | EndOfLine
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit
               | Operator | Delimiter
Keyword       = package | import | func | var | const | type | struct
               | if | else | for | return | int | float | bool | string
               | len | fmt | 'Print' | 'Println' | 'Scan'
Operator      = ArithOp | RelOp | LogicOp | AssignOp | IncDecOp | '&'
ArithOp       = '+' | '-' | '*' | '/' | '%' | '**'
LogicOp       = '&&' | '||' | '!'
Delimiter     = '(' | ')' | '[' | ']' | '{' | '}' | ',' | ';' | '.'

(* § Ідентифікатори та літерали *)
Ident         = Letter { Letter | Digit }
IntLit        = '0' | NonZeroDigit { Digit }
FloatLit      = Digits '.' [ Digits ] [ Exponent ]
               | Digits Exponent
               | '.' Digits [ Exponent ]
Digits        = Digit { Digit }
Exponent      = ( 'e' | 'E' ) [ '+' | '-' ] Digits
StringLit     = '"' { StringChar } '"'
StringChar    = OrdinaryChar | EscapeSeq

```

```

EscapeSeq      = '\' ( 'n' | 't' | 'r' | '"' | '\'' )
OrdinaryChar   = ? будь-який символ Unicode, крім '"', '\', '\n' та '\r' ?
Comment        = '//' { CommentChar }
CommentChar    = ? будь-який символ Unicode, крім '\n' та '\r' ?

(* § Алфавіт *)
Alphabet       = Letter | Digit | SpecialChar | WhiteSpace | EndOfLine | OtherChar
Letter         = LowerLetter | UpperLetter | '_'
LowerLetter    = 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i'
               | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' | 'r'
               | 's' | 't' | 'u' | 'v' | 'w' | 'x' | 'y' | 'z'
UpperLetter    = 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' | 'H' | 'I'
               | 'J' | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R'
               | 'S' | 'T' | 'U' | 'V' | 'W' | 'X' | 'Y' | 'Z'
Digit          = '0' | NonZeroDigit
NonZeroDigit   = '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
SpecialChar    = '+' | '-' | '*' | '/' | '%' | '=' | '!' | '<' | '>'
               | '&' | '|' | ':' | ';' | ',' | '.' | '(' | ')' | '['
               | ']' | '{' | '}' | '"' | '\'
WhiteSpace     = ' ' | '\t'
EndOfLine      = '\n' | '\r' '\n'
OtherChar      = ? інший символ Unicode (лише в StringLit та Comment) ?

```